UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Faculty of Engineering, Built Environment and Information Technology

Fakulteit Ingenieurswese, Bou-omgewing en Inligtingtegnologie / Lefapha la Boetšenere, Tikologo ya Kago le Theknolotši ya Tshedimošo

**Lecture Notes**
Department of Computer Science

**Artificial Intelligence COS314**

Lecturer: T. Nyathi t.nyathi@up.ac.za

©2025 University of Pretoria

# Contents


**1 Chapter 1 - An Introduction to Artificial Intelligence** **1**
1.1 Introduction . . . 1
1.1.1 Search and Knowledge Representation . . . 1
1.2 Areas of Artificial Intelligence . . . 1
1.2.1 Game Playing . . . 2
1.2.2 Theorem Proving . . . 2
1.2.3 Expert Systems . . . 2
1.2.4 Natural Language Processing . . . 3
1.2.5 Modelling Human Performance . . . 3
1.2.6 Planning and Robotics . . . 3
1.2.7 Artificial Neural Networks . . . 4
1.2.8 Evolutionary Algorithms and Other Nature-Inspired Methodologies . . . 5
1.2.9 Agent-based Technology . . . 5
1.2.10 Uncertainty Reasoning . . . 5
1.3 Artificial Intelligence Languages . . . 5

**2 Chapter 2 – Uninformed Search** **6**
2.1 Introduction . . . 6
2.2 Classic AI Problems . . . 6
2.2.1 Traveling Salesman . . . 6
2.2.2 Towers of Hanoi . . . 6
2.2.3 8-Puzzle . . . 6
2.3 Problem Representation . . . 7
2.3.1 Towers Hanoi . . . 7
2.3.2 Eight-Puzzle . . . 7
2.4 Graphs versus Trees . . . 9
2.5 Depth First Search . . . 10
2.6 Depth First Search with Iterative Deepening . . . 11
2.7 Breadth First Search . . . 12
2.8 Differences Between Depth First Search and Breadth First Search . . . 13

**3 Chapter 3 Informed Search** **14**
3.1 Calculating Heuristics . . . 14
3.2 Best First Search . . . 15
3.3 Hill-Climbing . . . 17
3.4 Greedy Hill Climbing . . . 18
3.5 The A Algorithm . . . 20
3.6 The A*Algorithm . . . 20
3.7 Branch and Bound . . . 23

**4 Chapter 4 Combinatorial Problems** **23**
4.1 Combinatorial Problems . . . 24
4.2 Problem and Solutions . . . 24
4.3 Perturbative and Constructive Solutions . . . 24
4.4 Systematic and Local Search . . . 25
4.5 Local Search . . . 25
4.6 Definition of a Local Search . . . 25
4.7 Characteristics of Local Search . . . 26
4.8 Simulated Annealing . . . 28
4.8.1 Disadvantages of Simulated Annealing . . . 29
4.9 Tabu Search . . . 29
4.9.1 Functionality . . . 29

4.9.2 Disadvantages of Tabu Search . . . 30

**5 Game and Game Playing** **31**
5.1 Search In-Game Playing . . . 31
5.2 Minimax Search Algorithm . . . 31
5.3 Characteristics of Minimax . . . 34
5.4 Alpha Beta Pruning . . . 35

**6 Metaheuristics** **40**
6.1 Introduction . . . 40
6.2 Metaheuristics Characteristics . . . 40
6.3 Classification of Metaheuristics . . . 40
6.4 Single Point . . . 40
6.4.1 Iterated Local Search . . . 41
6.4.2 Case Study - Knapsack Problem . . . 41

**7 Multipoint Search** **43**

**8 Genetic Algorithm** **44**
8.1 Initial Population Generation . . . 45
8.2 Fitness Evaluation . . . 45
8.3 Selection . . . 45
8.4 Crossover . . . 45
8.4.1 Mutation . . . 46
8.4.2 Population Update . . . 46
8.4.3 Termination . . . 46

**9 Genetic Programming** **46**
9.1 GP Algorithm . . . 47
9.1.1 Initial Population Generation . . . 48
9.1.2 Fitness Function . . . 48
9.1.3 Selection . . . 48
9.1.4 Genetic Operators . . . 48
9.1.5 Population Replacement . . . 49
9.1.6 Termination . . . 50
9.2 Applications of Genetic Programming . . . 50

**10 Grammatical Evolution** **50**
10.1 Introduction . . . 50
10.2 GE Algorithm . . . 51
10.2.1 Initial Population Generation . . . 51
10.2.2 Mapping . . . 52
10.2.3 Selection . . . 52
10.2.4 Crossover . . . 52
10.2.5 Mutation . . . 52
10.2.6 Population Replacement . . . 52
10.2.7 Termination . . . 52

**11 Swarm Intelligence** **53**
11.1 Introduction . . . 53
11.2 Ant Colony Optimisation . . . 53
11.3 Artificial Ants . . . 53
11.4 ACO Parameters . . . 55
11.5 Variations of ACO . . . 56
11.6 Applications of ACO . . . 56

11.7 Particle Swarm Optimisation . . . . . 56
11.7.1 Velocity Update . . . . . 57
11.7.2 Position Update . . . . . 58
11.7.3 PSO Parameters . . . . . 59
11.7.4 Variations of PSO . . . . . 59

**12 Adaptation and Hybrid Methods** **60**
12.1 Adaptation . . . . . 60
12.2 MultiObjective Optimisation . . . . . 60
12.3 Hybrid Algorithms . . . . . 60
12.4 Hybridisation of Evolutionary Algorithms . . . . . 61

**A References** **61**

# 1 Chapter 1 - An Introduction to Artificial Intelligence

## 1.1 Introduction

The field of artificial intelligence focuses on training computers to perform complex and time-intensive manual and cognitive tasks typically done by humans, such as those executed by machines in industrial settings. In 1956, a group of scientists convened at a conference to explore the concept of using computers to replicate human intelligence. During this early phase, fundamental questions were raised, including ”What constitutes intelligence?” ”Are machines capable of thought?” ”Can machines experience emotions?”, and ”What defines a machine?” These discussions led to the establishment of Artificial Intelligence (AI) as a discipline. Initial efforts to develop intelligent computer systems centred on activities like playing games and proving mathematical theorems. Additionally, researchers have worked on enabling computers to engage in common-sense reasoning.

To evaluate a computer’s intelligence, Alan Turing invented the Imitation Game. This test involved three participants: two humans and a computer, each isolated in separate rooms. One human, designated as the interrogator, was tasked with identifying which of the other two participants was the computer and which was human. If the computer could successfully mislead or persuade the interrogator into believing it was human, it would be deemed intelligent. This evaluation method later became known as the Turing Test. In 1937, addressing the questions ”What is a machine?” and ”Can machines think?”, Turing created the ”Turing Machine” to simulate the operations of a digital computer.

John Searle developed the Chinese Room thought experiment, building upon the concepts of the Turing test and the Turing machine. This experiment involves a non-Chinese speaker situated in a room filled with documents containing instructions and correlations. A Chinese individual outside the room slips a note with a Chinese request through an opening. The person inside uses the provided materials to craft a Chinese response, which is then passed back through the opening. This setup essentially mimics a Turing machine and serves to demonstrate that a computer can execute tasks without possessing genuine comprehension or cognitive abilities comparable to human thought processes.

### 1.1.1 Search and Knowledge Representation

Two of the key processes of artificial intelligence are:

- **Search** This technique solves a problem by searching a state space of possible solutions. Examples of search techniques include depth-first search, breadth-first search, heuristic searches, etc.
- **Knowledge Representation** refers to methods of representing knowledge that the computer needs to think intelligently and solve problems, e.g. semantic networks, predicate logic, etc. Both these processes form the basis of the different artificial intelligence areas.

## 1.2 Areas of Artificial Intelligence

Some of these areas include:

- Game Playing
- Theorem Proving
- Expert Systems
- Natural Language Understanding
- Modelling Human Performance
- Planning and Robotics

- Neural Networks
- Evolutionary algorithms and other nature-inspired methods
- Agent-based Technology
- Uncertainty reasoning
- Generative AI

### 1.2.1 Game Playing

This process involves programming computers to engage in specific board games that necessitate cognitive abilities. The board games that have been subject to research include checkers, chess, and the 15-puzzle. A state space representing each stage of the game is generated. Subsequently, a search is conducted through this space to determine an optimal winning path. An illustrative example is Deep Blue, a chess program developed by IBM, which competed and beat Kasparov, the world chess champion.

Another example is DeepMind, which achieved significant advancements in AI with its AlphaGo program. Notably, AlphaGo Zero's strategies were autonomously developed, as it was trained without any data from human games. AlphaGo Zero successfully defeated its predecessor in merely three days, utilising less computational power than AlphaGo. In contrast, the original AlphaGo required months to acquire proficiency in gameplay. The initial milestone occurred in October 2015 when the program defeated the European Go champion Fan Hui 5-0, in a game of Go. This marked the first instance in which an AI surpassed a professional Go player. Previously, computers were only known to have played Go at the "amateur" level. AlphaGo Zero accomplished these feats by employing a novel form of reinforcement learning, wherein AlphaGo Zero becomes its instructor. As previously elucidated, the system initiates with a single neural network devoid of any knowledge about the game of Go. By integrating this neural network with a sophisticated search algorithm, it subsequently engages in self-play. As it accumulates game experience, the neural network is refined and optimised to predict moves and even the eventual outcome of the games. This enhanced neural network is then reintegrated with the search algorithm to generate a new, more robust version of AlphaGo Zero, and the process iterates. With each iteration, the system's performance improves, and the quality of the self-play games advances, resulting in increasingly accurate neural networks and progressively more powerful versions of AlphaGo Zero.

### 1.2.2 Theorem Proving

One of the earliest domains addressed in the field of artificial intelligence is that of automatic mathematical theorem proving. Systems developed for this purpose generate proofs for simple theorems. Mathematical logic serves as the foundation for these systems. Newell and Simon's General Problem Solver represents one of the earlier systems developed in this field.

### 1.2.3 Expert Systems

An expert system is fundamentally a computer system that performs the tasks of a particular human expert, such as a physician. Knowledge from an expert is stored by the system and utilised to solve similar problems. Knowledge obtained from an expert is encoded into the system by an artificial intelligence specialist. Initially, the expert evaluates output from the system, and adjustments are made to the system accordingly. Expert Systems that have contributed significantly to this field include: DENDRAL, MYCIN, EMYCIN, PROSPECTOR and XCON. DENDRAL was one of the earliest systems developed. The formulas and mass spectrographic information about chemical bonds are entered into the system. Based on these, DENDRAL infers the structure of the organic molecules. MYCIN and EMYCIN are utilised to diagnose infectious diseases. XCON has been employed for configuring

VAX machines. Expert systems are suitable for specialised fields that have clearly defined problem-solving strategies. Domains in which they have been applied include medicine, education, business, design, and science.

### 1.2.4 Natural Language Processing

Research in Natural Language Processing (NLP) is focused on developing systems that can understand, interpret, and generate human language. This field has experienced significant advancements in recent years, progressing beyond early rule-based systems to sophisticated statistical and deep learning approaches. Deep learning architectures such as recurrent neural networks (RNNs) and transformers have revolutionised NLP. These models can learn complex relationships within language data, enabling tasks such as machine translation, text summarisation, and sentiment analysis with high accuracy.

Large Language Models (LLMs) are extensive neural networks trained on vast datasets of text and code. These models can perform various NLP tasks, including generating diverse creative text formats, translating languages, producing different types of creative content, and providing informative responses to queries. Examples include ChatGPT, Gemini, Deepseek, Jurassic-1 Jumbo, and Megatron-Turing NLG.

While earlier models such as SHRDLU and ELIZA were groundbreaking for their time, contemporary NLP techniques offer significantly more sophisticated capabilities. Deep learning models can process complex language structures, ambiguities, and nuances that were beyond the scope of earlier rule-based systems.

NLP research continues to advance the boundaries of possibility. As models and techniques evolve, it is anticipated that even more advanced applications will emerge, capable of understanding and interacting with human language in increasingly natural and meaningful ways.

### 1.2.5 Modelling Human Performance

This involves the creation of systems that model specific aspects of human problem-solving capabilities. An example of such a system is an Intelligent Tutoring System (ITS). An ITS aims to replicate the function of a human tutor by providing individualised instruction and feedback to learners. ITS have been developed across various disciplines, including programming, mathematics, medicine, and geography. Early instances of ITS include systems such as GUIDON, which was designed to tutor medical students.

Numerous ITS now utilise AI technologies such as machine learning and natural language processing to personalise learning experiences. This includes adaptive learning algorithms that adjust the pace and difficulty of instruction based on the learner's progress and performance. ITS can now analyse student data (e.g., learning styles, past performance, areas of difficulty) to provide customised feedback and recommendations. Interactive and engaging experiences: Contemporary ITS frequently incorporate interactive elements such as simulations, games, and virtual reality to enhance the immersive and engaging nature of learning. ITS are increasingly integrated with learning management systems (LMS) like **ClickUP** and other educational technologies to provide a seamless learning experience. Examples of contemporary ITS include Duolingo which is a language learning platform that employs AI and game-based elements to personalise lessons and provide interactive feedback. These advancements demonstrate the evolution of ITS into more sophisticated and effective tools for personalised and engaging learning experiences.

### 1.2.6 Planning and Robotics

Planning and Robotics Planning plays a crucial role in the development of flexible and responsive robots. It involves devising a sequence of actions that enable a robot to achieve a high-level goal, such as navigating a complex environment, manipulating objects, or collaborating with humans. Research in planning and robotics has significantly contributed to the field of agent-based systems and artificial

intelligence (AI). By developing sophisticated planning algorithms, researchers have enabled robots to operate autonomously in the following ways:

- Plan and execute complex tasks with minimal human intervention.
- Adapt to dynamic environments and unexpected situations.
- Collaborate effectively with humans, comprehending and responding to human intentions.

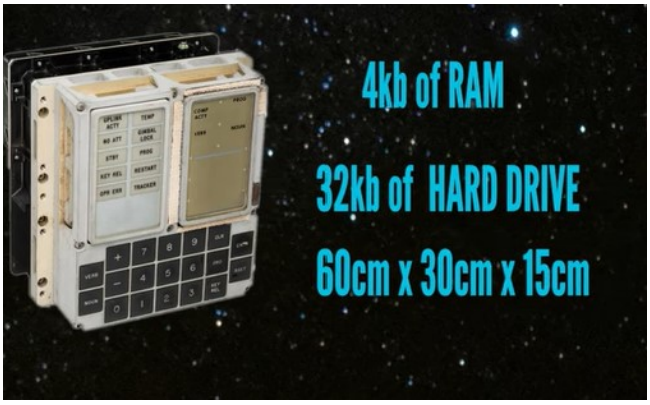


Figure 1: Fourth Astronaut

Figure 1 is the computer that sent humankind to the moon.

### 1.2.7 Artificial Neural Networks

This area investigates the use of models that have the same architecture as neurons in the human brain to solve problems. Neural network systems are good at interpreting noisy data. The architecture of a neuron is illustrated in 2

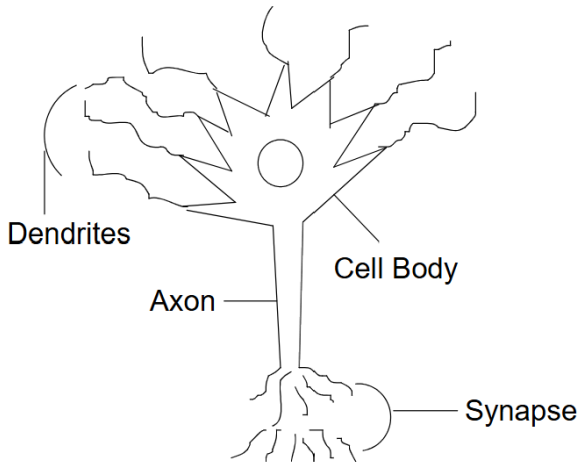


Figure 2: Biological Neuron

The dendrites represent weighted inputs to the neuron. An activation function is applied to the weighted input, the result of which is compared to a threshold value. This comparison results in a synapse (connection between two neurons) being excitatory or inhibitory. An excitatory synapse results in a signal being passed to the connected neuron. An inhibitory synapse results in a signal not being passed to the connected neuron.

### 1.2.8 Evolutionary Algorithms and Other Nature-Inspired Methodologies

Evolutionary algorithms are based on Darwin's theory of evolution. These algorithms essentially find a solution by first creating an initial population and then evolving successive generations that are fitter than the initial population. It is assumed that one of the successive generations contains a solution. Genetic algorithms and genetic programming are commonly used in AI applications. Two other nature-inspired methodologies are ant colonisation and swarm intelligence. Ant colonisation mimics the behaviour of ants in solving problems. Swarm intelligence uses swarming and flocking behaviours to solve problems. Similar to neural networks, these methods are also considered to be machine-learning techniques.

### 1.2.9 Agent-based Technology

Intelligent agents are used to perform mundane tasks such as searching through a list of email messages and sorting them in order of priority, or converting listings produced by the browser into a more specific and relevant list of sites. In a multi-agent system, agents communicate via an agent communication language.

### 1.2.10 Uncertainty Reasoning

In certain scenarios, a system may be required to process uncertain terms rather than precise information. For instance, the representation of terms such as "big" or "small" within a system presents a challenge. In such cases, uncertainty reasoning methods must be employed. The most prevalent methods utilised are fuzzy logic, Bayesian reasoning and networks.

## 1.3 Artificial Intelligence Languages

One of the tasks involved in developing a computer system is writing and implementing the required code. The programming language utilised for this purpose is dependent on the paradigm that is most suitable for the task at hand. Thus far, individuals have been exposed to two programming paradigms: the procedural programming paradigm (Pascal, Basic), and the object-oriented programming paradigm (Java, C++).

Initially, these paradigms were deemed inappropriate for the implementation of artificial intelligence problems. This led to the development of Prolog and Lisp, which have been the principal AI languages. The Prolog programming language supports the declarative programming paradigm. Prolog stands for Programming Logic and is based on mathematical formalisms, more specifically, first-order predicate logic. The Lisp (List Processing) programming language represents the functional programming paradigm. Recursive function theory forms the basis of this language. These programming languages originally interpreted code rather than compiled code. However, more recent versions of these programming languages include compilers. As research in the area of artificial intelligence matured, the reliance on Prolog and Lisp was reduced, and other programming paradigms and hence languages have been utilised to implement AI systems. AI systems have been developed in numerous languages, including Smalltalk, C, C++, Java and Python.

# 2 Chapter 2 – Uninformed Search

## 2.1 Introduction

This section examines the concept of a state space and the various search methods that can be employed to explore the search space to identify a solution. Before solving an artificial intelligence problem, it must be represented as a state space. Subsequently, the state space is searched to determine a solution to the problem. A state space fundamentally comprises a set of nodes representing each state of the problem, arcs between nodes representing the permissible transitions from one state to another, an initial state, and a goal state. Each state space manifests in the form of a tree or a graph. The selection of an appropriate search algorithm or technique is contingent upon various factors, including the nature of the problem and its potential representation. The course curriculum will encompass an examination of the following search techniques:

- Depth First Search
- Depth First Search with Iterative Deepening
- Breadth-First Search
- Best First Search
- Hill Climbing
- Branch and Bound Techniques
- A* Algorithm

## 2.2 Classic AI Problems

Three classic AI problems referred to in this section are the Travelling Salesman problem, the Towers of Hanoi problem, and the Eight Puzzles.

### 2.2.1 Traveling Salesman

A salesman has a list of cities that he must visit exactly once. There are direct roads between each pair of cities on the list. Find the route the salesman should follow for the shortest trip that starts and finishes at any city.

### 2.2.2 Towers of Hanoi

In a monastery situated in the remote regions of Tibet, there exist three crystal columns and 64 golden rings. The rings vary in size and are positioned on the columns. At the inception of time, all rings were situated on the leftmost column, and since then, the monks have been tasked with continuously relocating the rings individually between the columns. When transferring the rings, it is imperative that a larger ring not be placed upon a smaller ring. Additionally, only one ring may be transferred from one column to another at a time. A simplified version of this problem under consideration involves only two or three rings instead of 64.

### 2.2.3 8-Puzzle

The 8-puzzle involves the manipulation of tiles on a board to achieve a specific configuration. The black square on the board represents a space. A player can relocate a tile into this space, thereby creating a new vacant position for subsequent tile movement. For instance, given the initial state described in Figure 3, the objective may be to rearrange the tiles to attain the goal state described in Figure 4.

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Figure 3: 8 Puzzle Initial State

| | | |
|---|---|---|
| 1 | 8 | 3 |
| 2 | 6 | 4 |
| 7 | | 5 |

Figure 4: 8 Puzzle 1 Move

### 2.3 Problem Representation

Several factors need to be taken into consideration when developing a state space representation. Factors that must be addressed are:

- What is the goal to be achieved?
- What are the legal moves or actions?
- What knowledge needs to be represented in the state description?
- Classification of problems - There exist three primary categories of problems. Certain problems necessitate only a representation, such as crossword puzzles. Other problems require a binary response indicating the presence or absence of a solution. The final category comprises problems that demand a solution path as output, exemplified by mathematical theorems and the Towers of Hanoi problem. In these instances, the goal state is known, and the objective is to determine the method of attaining this state.
- Best solution vs. Good enough solution - For certain problems, a satisfactory solution is adequate. For instance, theorem proving and the eight queens problem. However, some problems necessitate an optimal or best solution, such as the travelling salesman problem.

#### 2.3.1 Towers Hanoi

A possible state space representation of the Towers of Hanoi problem using a graph is indicated in Figure 5. The legal moves in this state space involve moving one ring from one pole to another, moving one ring at a time and ensuring that a larger ring is not placed on a smaller ring.

#### 2.3.2 Eight-Puzzle

Although a player moves the tiles around the board to change the configuration of tiles. However, we will define the legal moves in terms of moving the space. The space can be moved up, down, left and right. This is shown in Figure 6. Summary:

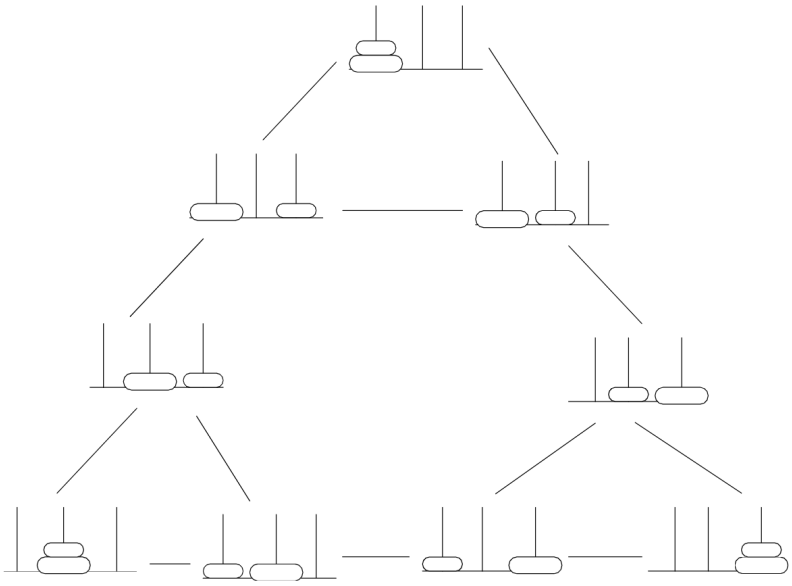

Figure 5: Towers of Hanoi state space representation

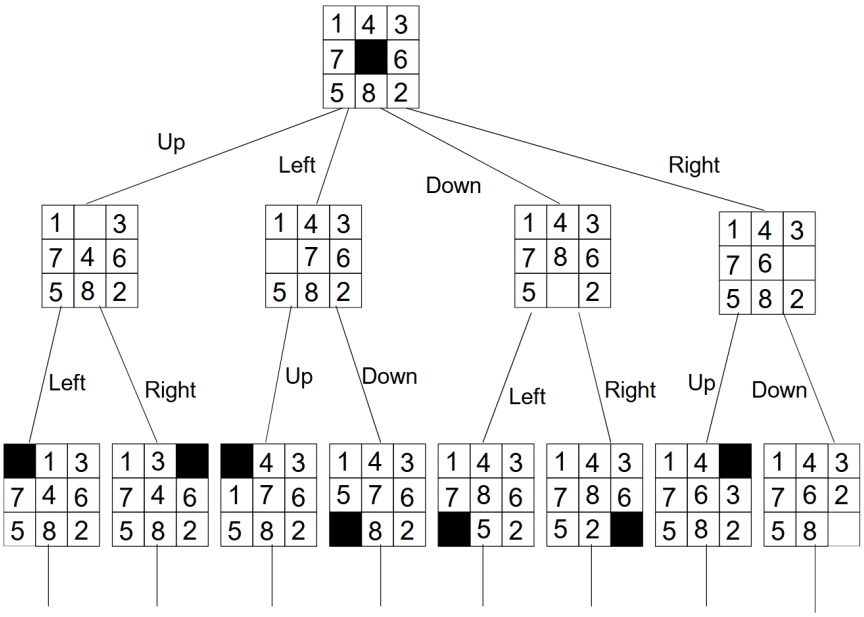


Figure 6: Eight-Puzzle Problem state space representation

- A state space is defined as a set of descriptions or states.
- Each search problem consists of: One or more initial states. A set of legal actions. Actions are represented by operators or moves applied to each state. For example, the operators in a state space representation of the 8-puzzle problem are left, right, up, and down. One or more goal states.
- The number of operators is problem-dependent and specific to a particular state-space representation. The greater the number of operators, the larger the branching factor of the state space. Thus, the number of operators should be kept to a minimum, for example, the 8-puzzle: operations are defined in terms of moving the space instead of the tiles.
- Why generate the state space at run-time, and not just have it built in advance?
- A search algorithm is applied to a state space representation to find a solution path. Each search algorithm applies a particular search strategy.

## 2.4 Graphs versus Trees

If states in the solution space can be revisited more than once, a directed graph is used to represent the solution space. In a graph, more than one move sequence can be obtained from one state to another. The moves in the graph can be undone. In a graph, there is more than one path to a goal, whereas in a tree, the path is more clearly distinguishable. A goal state may have to appear more than once in a tree. Search algorithms for graphs must consider possible loops and cycles in the graph. Trees may be more "efficient" for representing problems such as loops and cycles that do not have to be catered for. The entire tree or graph is not generated.

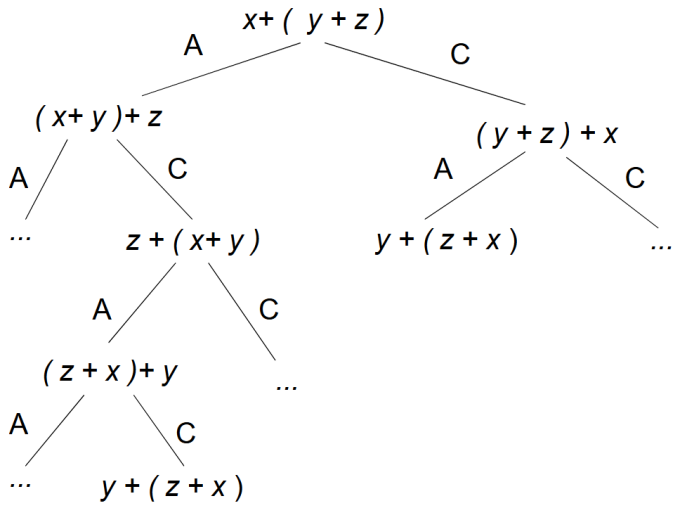


Figure 7: Graph vs Tree

Notice that there is more than one path connecting two particular nodes in a graph, whereas this is not so in a tree.

**Example:**
Prove x + (y +z) = y + (z+x) given
L+(M+N) = (L+M) + N .....(A)
M+N = N+M.....(C)

Figure 8 represents the corresponding state space as a graph. The same state space is represented as a tree in Figure 9. Notice that the goal state is represented more than once in the tree.

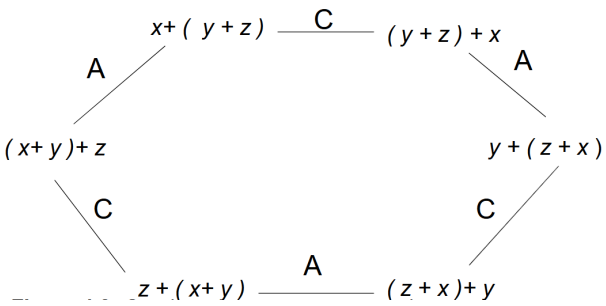


Figure 8: Graph state space representation

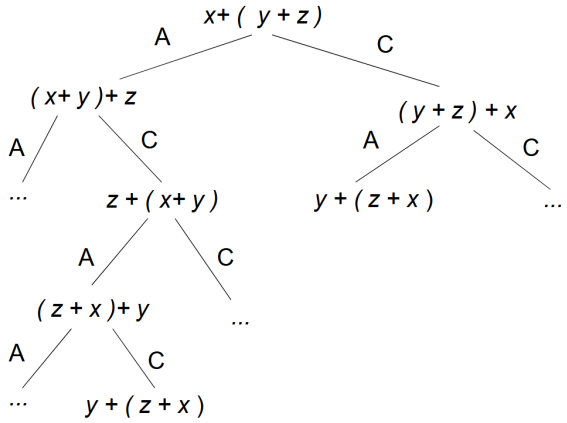


Figure 9: Tree state space representation

## 2.5 Depth First Search

One of the searches commonly used to search a state space is the depth-first search. The depth-first search searches to the lowest depth or ply of the tree. Consider the tree in Figure 10. In this case, the tree has been generated before the search.

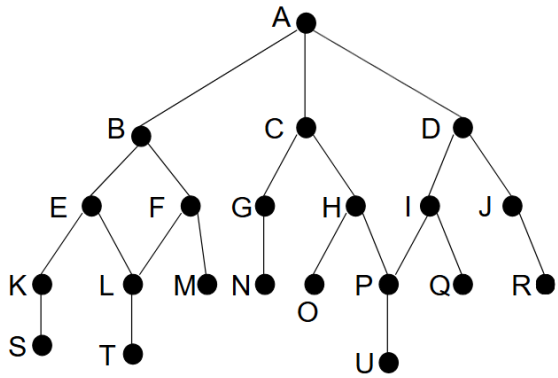


Figure 10: Depth First Search

Each node is not visited more than once. A depth-first search of this tree produces: A, B, E, K, S, L, T, F, M, C, G, N, H, O, P, U, D, I, Q, J, R. However, in this example, the tree was generated first, and then a search of the tree was conducted. However, often there is not enough space to generate the entire tree representing the state space and then search it. The algorithm presented below conducts a depth-first search and generates parts of the tree as it goes along.

```
Depth First Search Algorithm
def dfs(in Start, out State)
  open = [Start];
  closed = [];
  State = failure;
  while (open <> []) AND (State <> success)
  begin
    remove the leftmost state from open, call it X;
    if X is the goal, then
      State = success
    else begin
      generate children of X;
      put X on closed
      eliminate the children of X on open or closed
      put remaining children on left end of open
    end else
  endwhile
  return State;
enddef
```

- **Example 1**
  Suppose that the letters A, B, etc. represent states in a problem. The following moves are legal:
  A to B and C
  B to D and E
  C to F and G
  D to I and J
  I to K and L

  Start state: A Goal state: E, J

Let us now conduct a depth-first search of the space. Remember that the tree will not be generated beforehand, but is generated as part of the search. The search and hence Generation of the tree will stop as soon as success is encountered or the open list is empty.

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1. | A | - | | | failure |
| 2. | | | A | B, C | failure |
| 3. | B, C | A | | | failure |
| 4. | C | A | B | D, E | failure |
| 5. | D, E, C | A, B | | | failure |
| 6. | E, C | A, B | D | I, J | failure |
| 7. | I, J, E, C | A, B, D | | | failure |
| 8. | J, E, C | A, B, D | I | K, L | failure |
| 9. | K, L, J, E, C | A, B, D, I | | | failure |
| 10. | L, J, E, C | A, B, D, I, | K | none | failure |
| 11. | J, E, C | A, B, D, I | L | none | failure |
| 12. | E, C | A, B, D, I | J | | success |

Figure 11: DFS Table

## 2.6 Depth First Search with Iterative Deepening

Iterative deepening involves performing the depth-first search on several different iterations with different depth bounds or cutoffs. On each iteration, the cutoff is increased by one. The iterative process

terminates when a solution path is obtained. Iterative deepening is more advantageous in cases where the state-space representation has infinite depth rather than constant depth. The state space is searched at a deeper level in each iteration. DFID is guaranteed to find the shortest path. The algorithm retains information between iterations; thus, prior iterations are "wasted."

```
procedure DFID (intial_state, goal_states)
begin
       search_depth=1
    while (solution path is not found)
    begin
        dfs(initial_state, goals_states) with a depth bound of search_depth
       increment search_depth by 1
    endwhile
end
```

- **Example 1** :
  Suppose that the letters A, B, etc represent states in a problem. The following moves are legal:
  A to B and C
  B to D and E
  C to F and G
  D to I and J
  I to K and L

  Start state: A Goal state: E, J

**Iteration 1: search_depth =1**

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1. | A | - | | | failure |
| 2. | | | A | B, C | failure |
| 3. | B, C | A | | | failure |
| 4. | C | A | B | | failure |
| 5. | C | A, B | | | failure |
| 6. | | A, B | C | | failure |
| 7. | | A, B, C | | | failure |

**Iteration 2: search_depth = 2**

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1. | A | - | | | failure |
| 2. | | | A | B, C | failure |
| 3. | B, C | A | | | failure |
| 4. | C | A | B | D, E | failure |
| 5. | D, E, C | A, B | | | failure |
| 6. | E, C | A, B | D | | failure |
| 7. | E, C | A, B, D | | | failure |
| 8. | C | A, B, D | E | | success |

## 2.7 Breadth First Search

The breadth-first search visits nodes left-to-right at each level of the tree. Each node is not visited more than once. A breadth-first search of the tree in Figure 10 produces A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U. Although in this example the tree was generated first and then a search of the tree was conducted. However, often there is not enough space to generate the entire

tree representing the state space and then search it. The algorithm presented below conducts a breadth-first search and generates parts of the tree as it moves.

```
def bfs (in Start, out State)
  open = [Start];
  closed = [];
  State = failure;
  while (open <> []) AND (State <> success)
  begin
    remove the leftmost state from open, call it X;
    if X is the goal, then
      State = success
    else begin
      generate children of X;
      put X on closed
      eliminate the children of X on open or closed
      put remaining children on right end of open
    end else
  endwhile
  return State;
enddef
```

Let us apply the breadth-first search to the problem in Example 1 in this section.

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1. | A | - | | | failure |
| 2. | | | A | B, C | failure |
| 3. | B, C | A | | | failure |
| 4. | C | A | B | D, E | failure |
| 5. | C, D, E | A, B | | | failure |
| 6. | D, E | A, B | C | F, G | failure |
| 7. | D, E, F, G | A, B, C | | | failure |
| 8. | E, F, G | A, B, C | D | I, J | failure |
| 9. | E, F, G, I, J | A, B, C, D | | | failure |
| 10. | F, G, I, J | A, B, C, D | E | | success |

Figure 12: Breadth First Search

## 2.8 Differences Between Depth First Search and Breadth First Search

Breadth-first is guaranteed to find the shortest path from the start to the goal. DFS may get "lost" in search, and hence a depth-bound may have to be imposed on a depth-first search. BFS has a bad branching factor,r, which can lead to combinatorial explosion. The solution path found by DFS may be long and suboptimal. DFS is more efficient for search spaces with many branches, i.e. the branching factor is high. The following factors need to be taken into consideration when deciding which search to use:

- The importance of finding the shortest path to the goal.
- The branching of the state space.
- The available time and resources.
- The average length of paths to a goal node.
- All solutions or only the first solution.

# 3 Chapter 3 Informed Search

DFS and BFS may require too much memory to generate the entire state space. In these cases, a heuristic search is performed. Heuristics help reduce the size of the search space. An evaluation function is applied to each goal to assess how promising it is in leading to the goal. Examples of heuristic searches:

- Best first search
- A* algorithm
- Hill climbing

Heuristic searches incorporate the use of domain-specific knowledge in the process of choosing which node to visit next in the search process. Search methods that include the use of domain knowledge in the form of heuristics are described as “weak” search methods. The knowledge used is “weak” in that it usually helps, but does not always help to find a solution.

## 3.1 Calculating Heuristics

Heuristics are rules of thumb that may find a solution, but are not guaranteed. Heuristic functions are also defined as evaluation functions that estimate the cost from a node to the goal node. The incorporation of domain knowledge into the search process using heuristics is intended to speed up the search process. However, heuristic functions are not guaranteed to be accurate. If they were, there would be no need for a search process. Heuristic values are greater than or equal to zero for all nodes. Heuristic values are the approximate costs of finding a solution. A heuristic value of zero indicates that the state is the goal state. A heuristic that never overestimates the cost to the goal is referred to as an admissible heuristic. Not all the heuristics are admissible. A heuristic value of infinity indicates that the state is a “dead-end” and is not going to lead anywhere. A good heuristic does not require a long computation time. Heuristics are often defined as a simplified or relaxed version of the problem, for example, the number of tiles that are out of place. A heuristic function $h_1$ is better than some heuristic functions $h_2$ if fewer nodes are expanded during the search when $h_1$ is used than when $h_2$ is used. Each of the following shown in Figure 14, could serve as a heuristic function for the

Initial State

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | | 5 |

Goal State

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Figure 13: The 8-puzzle problem

8-Puzzle problem:

- Number of tiles out of place - count the number of tiles out of place in each state compared to the goal.
- Sum all the distances that the tiles are out of place.
- Tile reversals - multiply the number of tile reversals by 2.

| State | Tiles out of place | Sum of distances out of place | 2 x the number of direct tile reversals |
|---|---|---|---|
| 2 8 3<br>1 6 4<br>7 5 | 5 | 6 | 0 |
| 2 8 3<br>1 4<br>7 6 5 | 3 | 4 | 0 |
| 2 8 3<br>1 6 4<br>7 5 | 5 | 6 | 0 |

Figure 14: Heuristic functions for the 8-puzzle problem

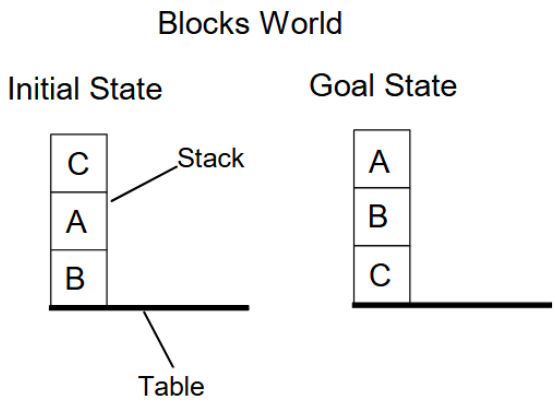


Figure 15: Blocks World

**Exercise:** Derive a heuristic function for the Block World problem shown in Figure 15 The Blocks World problem is a classic planning problem in artificial intelligence used to illustrate various search and planning algorithms. It involves a set of blocks of different sizes and colours and a table (or another surface) on which they can be stacked. The goal is to rearrange the blocks from an initial configuration to a desired goal configuration, following certain rules or constraints.

Experience has shown that it is difficult to devise heuristic functions. Furthermore, heuristics are fallible and are by no means perfect.

## 3.2 Best First Search

Figure 16 illustrates the application of the best-first search on a tree that had been generated before the algorithm was applied

The best-first search is a general search where the minimum cost nodes are first expanded. The best-first search is not guaranteed to find the shortest solution path. It is more efficient not to revisit nodes (unless you want to find the shortest path). This can be achieved by restricting legal moves to not include nodes that have already been visited. The best-first search attempts to minimise the cost of finding a solution. It is a combination of depth-first search and breadth-first search with heuristics. Two lists are maintained in the implementation of the best-first algorithm: OPEN and CLOSED. OPEN contains nodes that have been evaluated by the heuristic function but have not yet been expanded into successors. The CLOSED contains nodes that have already been visited. Application of best-

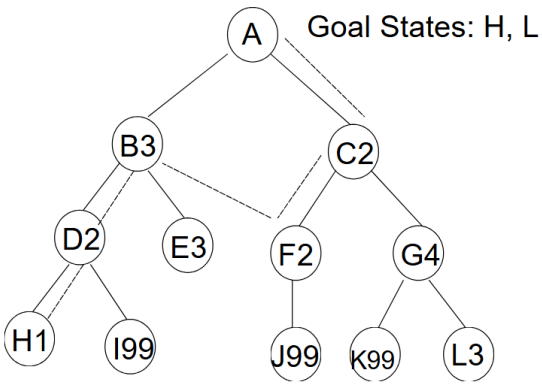


Figure 16: Best First Search

first search: Internet spiders. The algorithm for the best-first search is described as follows: The algorithm generates the tree/graph representing the state space.

```
def bestfs(in Start, out State)
open = [Start],
close = [],
State = failure;
     while (open <> []) AND (State <> success)
        begin
            remove the leftmost state from open, call it X;
            if X is the goal, then
            State = success
            else begin
            generate children of X;
            for each child of X do
            case
            the child is not on open or closed
            begin
            assign the child a heuristic value,
            add the child to open,
            end;
            the child is already on open
            if the child was reached by a shorter path then
            give the state on open the shorter path
            the child is already on closed:
            if the child is reached by a shorter path then
            begin
            remove the state from closed,
            add the child to open
            end;
            endcase
        endfor
        put X on closed;
        re-order states on open by heuristic merit (best leftmost);
    endwhile;
return State;
end.
```

We will now apply the best first search to a problem.

- **Example 1:** Suppose that each letter represents a state in a solution space. The following moves are legal:
  A5 to B4 and C4
  B4 to D6 and E5
  C4 to F4 and G5
  D6 to I7 and J8
  I7 to K7 and L8

  Start state: A Goal state: E

  The numbers next to the letters represent the heuristic value associated with that particular state.

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1 | A5 | - | | | failure |
| 2 | | | A5 | B4, C4 | failure |
| 3 | B4, C4 | A5 | | | failure |
| 4 | C4 | A5 | B4 | D6, E5 | failure |
| 5 | C4, E5, D6 | A5, B4 | | | failure |
| 6 | E5, D6 | A5, B4 | C4 | F4, G5 | failure |
| 7 | F4, E5, G5, D6 | A5, B4, C4 | | | failure |
| 8 | E5, G5, D6 | A,5 B4, C4 | F4 | none | failure |
| 9 | E5, G5, D6 | A5, B4, C4, F4 | | | failure |
| 10 | G5, D6 | A5, B4, C4, F4 | E5 | | success |

Figure 17: Best-first search

### 3.3 Hill-Climbing

Hill climbing is similar to best-first search. While the best first search considers states globally, hill-climbing considers only local states. The application of the hill-climbing algorithm to a tree that has been generated before the search is, as illustrated in Figure 18. The hill-climbing algorithm is

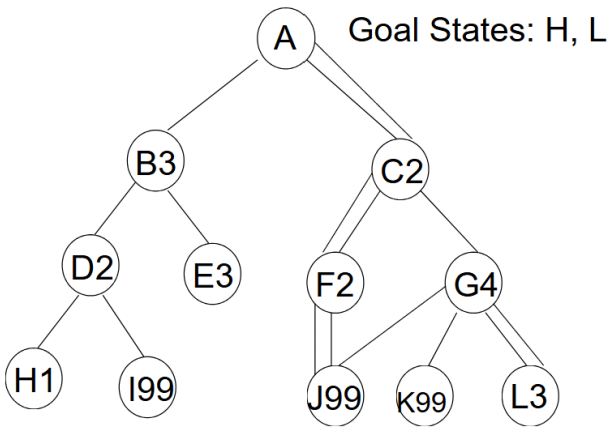


Figure 18: Hill Climbing

described below. The hill-climbing algorithm generates a partial tree/graph.

```
def hill_climbing(in Start, out State)
open = [Start], close = [],
```

```
State = failure;
while (open <> []) AND (State <> success)
begin
remove the leftmost state from open, call it X;
if X is the goal, then
State = success
else begin
generate children of X;
for each child of X do
case
the child is not on open or closed
begin
assign the child a heuristic value,
end;
the child is already on open
if the child was reached by a shorter path then
give the state on open the shorter path
the child is already on closed:
if the child is reached by a shorter path then
begin
remove the state from closed,
end;
endcase
endfor
put X on closed;
re-order the children states by heuristic merit (best leftmost);
place the reordered list on the leftmost side of open;
endwhile;
return State;
end.
```

- **Example 1**: Suppose that each of the letters represents a state in a state space representation.
  Legals moves:
  A to B3 and C2
  B3 to D2 and E3
  C2 to F2 and G4
  D2 to H1 and I99
  F2 to J99
  G4 to K99 and L3

  Start state: A Goal state: H, L

The result of applying the hill-climbing algorithm to this problem is illustrated in the table below in Figure 19.

### 3.4 Greedy Hill Climbing

This is a form of hill climbing which chooses the node with the least estimated cost at each state. However, this does not guarantee an optimal solution path. Greedy hill climbing is susceptible to the problem of becoming stuck in local optima.
**The Greedy Hill Climbing Algorithm**

1. Evaluate the initial state.

| | OPEN | CLOSED | X | X's Children | State |
|---|---|---|---|---|---|
| 1 | A | - | | | failure |
| 2 | | | A | B3, C2 | failure |
| 3 | C2, B3 | A | | | failure |
| 4 | B3 | A | C2 | F2, G4 | failure |
| 5 | F2, G4, B3 | A, C2 | | | failure |
| 6 | G4, B3 | A, C2 | F2 | J99 | failure |
| 7 | J99, G4, B3 | A, C2, F2 | | | failure |
| 8 | G4, B3 | A, C2, F2 | J99 | none | failure |
| 9 | G4, B3 | A, C2, F2, J99 | | | failure |
| 10 | B3 | A, C2, F2, J99 | G4 | K99, L3 | failure |
| 11 | L3,K99, B3 | A, C2, F2,J99, G4 | | | failure |
| 12 | K99, B3 | A, C2, F2, J99, G4 | L3 | | success |

Figure 19: Hill climbing

2. Select a new operator.
3. Evaluate the new state.
4. If it is closer to the goal state than the current state, make it the current state. If it is not better,r ignore.
5. If the current state is the goal state or no new operators are available, quit.
6. Otherwise, repeat steps 2 to 4.

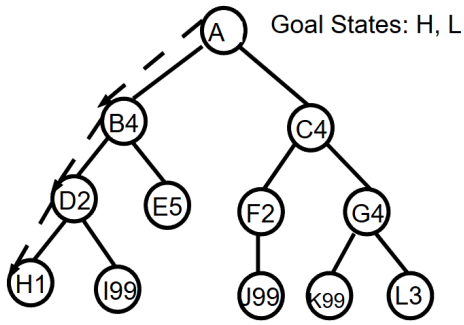


Figure 20: Greedy Hill Climbing without Backtracking

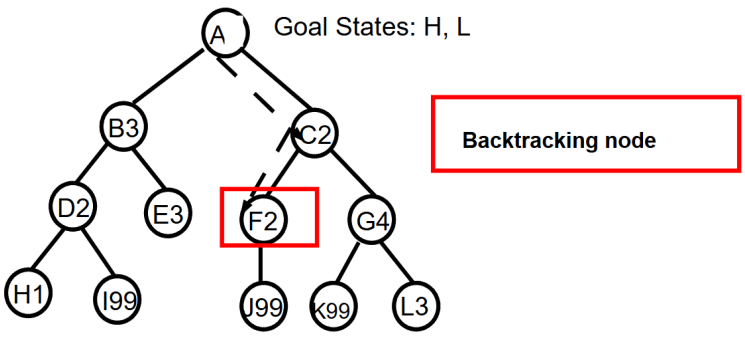


Figure 21: Greedy Hill Climbing with Backtracking 1

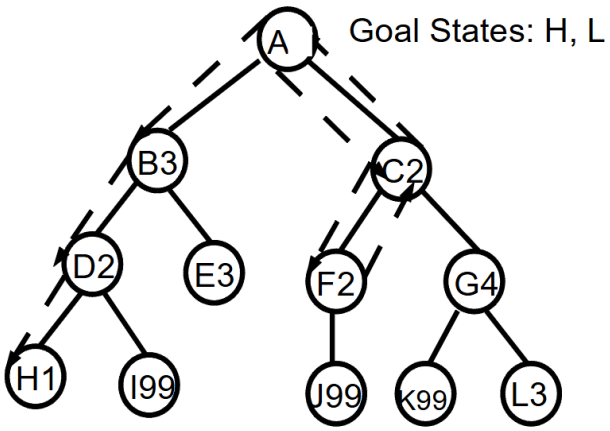


Figure 22: Greedy Hill Climbing with Backtracking 2

### 3.5 The A Algorithm

The A algorithm is essentially the best first search implemented with the following function:

$$f(n) = g(n) + h(n) \tag{1}$$

where g(n) - measures the length of the path from any state n to the start state
h(n) - is the heuristic measure from the state n to the goal state.
The use of the evaluation function f(n) applied to the 8-Puzzle problem is illustrated in the following image of the initial and goal states. The heuristic used for h(n) is the number of tiles out of place. The

Initial State

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | | 5 |

Goal State

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Figure 23: Enter Caption

numbers next to the letters represent the heuristic value associated with that particular state.

### 3.6 The A*Algorithm

Search algorithms that are guaranteed to determine the shortest path are called admissible algorithms. Breadth-first search is an example of an admissible algorithm. The evaluation function we have considered with the best first algorithm is

$$f(n) = g(n) + h(n), \tag{2}$$

where
g(n) - is an estimate of the cost of the distance from the start node to some node n, e.g. the depth at which the state is found in the graph.
h(n) - is the heuristic estimate of the distance from n to a goal.
An evaluation function used by admissible algorithms is:

$$f^*(n) = g^*(n) + h^*(n) \tag{3}$$

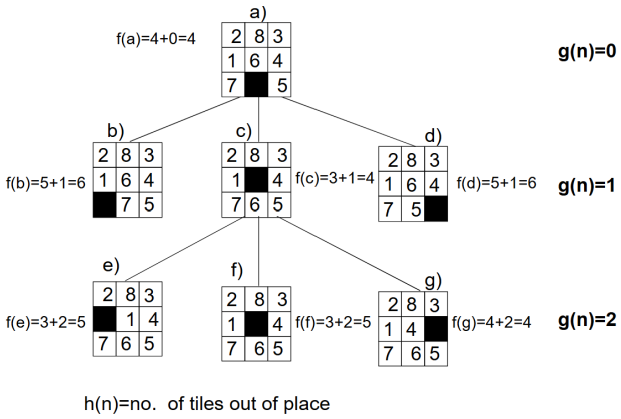


Figure 24: 8 puzzle

where
$g^*$(n) - estimates the shortest path from the start node to node n. $h^*$(n) - estimates the actual cost from the start node to the goal node that passes through n. $f^*$ does not exist in practice, and we attempt to estimate f as closely as possible to $f^*$. g(n) is a reasonable estimate of $g^*$(n). Usually g(n) $\leq g^*$(n). It is usually not possible to compute $h^*$(n). Instead, we try to find a heuristic h(n) that is bounded from above by the actual cost of the shortest path to the goal, that is, h(n) $\leq h^*$(n). The best first search used with f(n) is called the A algorithm. If h(n) $\leq h^*$(n) in the A algorithm, then the A algorithm is called the $A^*$ algorithm. The heuristic that we developed for the 8-puzzle problem is bounded above by the number of moves required to move to the goal position. The number of tiles out of place and the sum of the distance from each correct tile position is less than the number of required moves to move to the goal state. Thus, the best first search applied to the 8-puzzle using these heuristics is an $A^*$ algorithm.

- **Exercise 1:** Consider the following state space. **G** is the goal state. Note that the values of g(n) are given on the arcs:

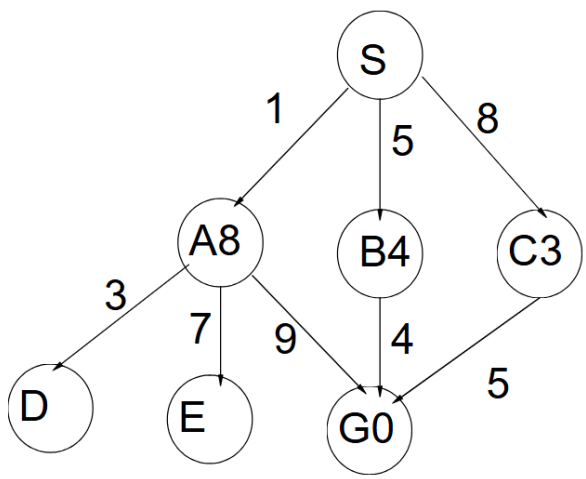


Figure 25: $A^*$ Algorithm Example

1. Is h(n) admissible?
2. What is the optimal solution path?
3. What is the cost of this path?

Note: Remember, we cannot see the entire state space when we apply the search algorithm.

- **Exercise 2**: Consider the following state space. G1 and G2 are goal states:

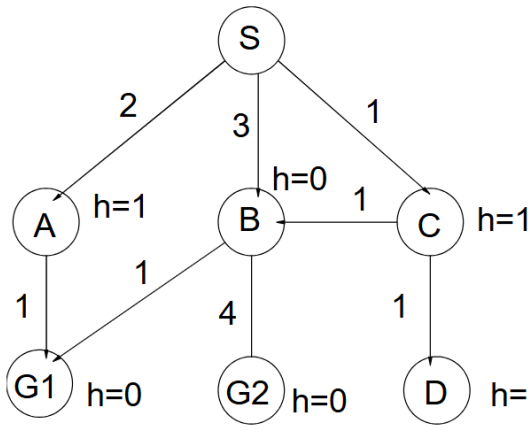


Figure 26: A* Algorithm Example 2

1. Is h(n) admissible?
2. What is the optimal solution path?
3. What is the cost of this path?

Note: Remember, we cannot see the entire state space when we apply the search algorithm. The A* search algorithm is a popular and powerful pathfinding algorithm that finds the shortest path between two points, using a combination of graph search and a heuristic estimate to prioritise its search.

**Advantages:**

- Optimality: A* guarantees finding the shortest path if the heuristic function used is admissible (never overestimates the true cost) and consistent (the estimated cost from a node to the goal is never greater than the cost of going to a neighbour and then to the goal).
- Completeness: If a solution exists, A* will find it, provided the graph doesn't have an infinite cost or number of nodes.
- Efficiency: A* is often more efficient than other search algorithms because it uses the heuristic to guide its search, exploring the most promising paths first. This reduces the number of nodes that need to be expanded.
- Widely Applicable: A* can be used in a variety of domains, including: Navigation and maps: Finding the shortest route between two locations.
  - Game AI: Helping characters navigate game worlds.
  - Robotics: Planning robot movements.
  - Network routing: Finding the most efficient path for data to travel across a network.

**Disadvantages:**

- Heuristic Dependency: The performance of A* heavily relies on the quality of the heuristic function. A poorly chosen heuristic can lead to:
- Increased search time: If the heuristic underestimates the cost, A* might explore many unnecessary paths.
- Suboptimal solutions: If the heuristic overestimates the cost, A* might miss the shortest path and find a suboptimal solution.

- Memory Consumption: A* can require significant memory to store the ”open” and ”closed” sets of nodes, especially for large graphs. The ”open” set contains nodes that have been generated but not yet explored, and the ”closed” set contains nodes that have already been explored.
- Complexity: Implementing A* correctly can be complex, as it requires careful management of data structures and the heuristic function.

### 3.7 Branch and Bound

Branch and bound techniques are used to find the optimal solution. Each solution path has a cost associated with it. Branch and bound techniques keep track of the solution and its cost and continue searching for a better solution further on. The entire search space is usually searched. Each path and hence each arc (rather than the node) has a cost associated with it. The assumption made is that the cost increases with path length. Paths that are estimated to have a higher cost than paths already found are not pursued. These techniques are used in conjunction with depth-first and breadth-first searches as well as iterative deepening.

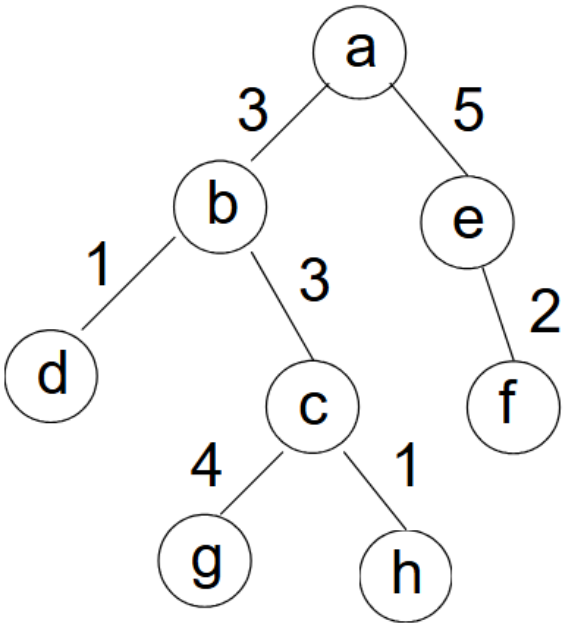


Figure 27: Branch and Bound

In this example, d, g, h and f represent the goal state.
A branch and bound technique applied to this tree will first find the path from a to d, which has a cost of 4. The technique will then compare the cost of each sub-path to 4 and will not pursue the path if the sub-path has a cost exceeding 4. Both the sub-paths a to e and a to c have a cost exceeding four; these sub-paths are not pursued any further. Branch and bound is a powerful technique for solving optimisation problems, particularly those that are combinatorial. It systematically explores the solution space, pruning away portions that are guaranteed not to contain the optimal solution.

## 4 Chapter 4 Combinatorial Problems

Combinatorial problems are mathematical puzzles that focus on counting, arranging, or choosing items in specific ways. These problems deal with a limited number of items, and the goal is to figure out how many different arrangements or selections are possible, given certain rules or restrictions. Examples include figuring out how many ways you can order a set of objects (permutations), how many ways you can pick a smaller group from a larger one (combinations), how many ways you can divide items among people (partitioning), how many ways you can color a map (graph coloring), or

how many different routes you can take through a network (graph traversal). These types of problems pop up in lots of different areas, like computer science, cryptography, game theory, and probability. They're also important for practical tasks like planning, scheduling, creating timetables, allocating resources, designing codes and hardware, and even analysing DNA. They involve finding the best way to group, order or assign a limited set of items while meeting certain requirements.

## 4.1 Combinatorial Problems

In combinatorial optimisation problems, the goal is to find an optimal solution that minimises or maximises an objective function, such as the total cost, distance, time, or profit. The solution space for these problems can be huge, and exhaustive search methods may not be practical or feasible. Therefore, combinatorial optimisation problems often require the use of specialised algorithms and techniques such as heuristics, metaheuristics, and dynamic programming to efficiently search the solution space and find near-optimal solutions. Some examples of combinatorial optimisation problems include the following.

- The travelling salesman problem, which seeks to find the shortest possible route that visits a given set of cities and returns to the starting city
- The knapsack problem seeks to find the optimal set of items to include in a knapsack, given a set of items with different weights and values and a limited carrying capacity.
- The job scheduling problem seeks to find the optimal schedule for completing a set of tasks or jobs, subject to resource and time constraints.
- The facility location problem seeks to find the optimal locations for a set of facilities to minimise transportation and operating costs.
- The SAT problem, short for Boolean Satisfiability Problem, is classic and fundamental in computer science and mathematical logic. Given a Boolean expression, can you assign values to the variables so that the entire formula evaluates to true?

Combinatorial optimisation problems have applications in many fields, including logistics, transportation, manufacturing, finance, and telecommunications.

## 4.2 Problem and Solutions

A problem is an abstract problem (problem class), such as" For any given set of location points in a Euclidean plane, find the shortest round trip. An example of this problem is to find the shortest round trip for a specific set of points in the plane. The solution to such a problem instance is a specific shortest round-trip connecting a given set of points. However, the solution of the abstract problem is an algorithm that, given a problem instance, determines a solution for that instance. In general, problems can be defined as sets of problem instances, where each instance is a pair of input and solution data. Candidate solutions are potential solutions that may be encountered during an attempt to solve a given problem instance; however, unlike solutions, they may not satisfy all the conditions from the problem definition. For our shortest round-trip example, typically any valid round-trip connecting the given set of points, regardless of length, would be a candidate solution, while only candidate round-trips with minimal length would qualify as solutions.

## 4.3 Perturbative and Constructive Solutions

Candidate solutions in combinatorial problems consist of solution components, such as assigning truth values to individual propositional variables (atomic assignments) in the SAT problem. Modifying one or more of these components can create new candidate solutions in a process known as **perturbing** a given solution. Search algorithms that rely on this mechanism to generate candidate solutions are known as **perturbative search methods**. Perturbative approaches typically involve

directly searching for candidate solutions in the solution space. However, it can also be beneficial to include partial candidate solutions in the search space, that is, candidate solutions in which one or more solution components are absent. The task of generating (complete) candidate solutions by iteratively extending partial candidate solutions can be formulated as a search problem in which typically the goal is to obtain a 'good' candidate solution. In contrast, for optimisation problems, the goodness corresponds to the value of the objective function. Algorithms for solving this type of problem are known as **constructive search methods** (or construction heuristics).

## 4.4 Systematic and Local Search

Search approaches can be classified as systematic or local. Systematic search algorithms navigate a problem instance's search space systematically to find an optimal solution or to determine with certainty that none exists. Completeness is a typical property of a systematic search algorithm. A complete algorithm is guaranteed to find the optimal solution if one exists or prove that no solution exists, in a finite amount of time. Conversely, local search algorithms begin at a particular point in the search space and move to a neighbouring location based on local knowledge. Each location in the search space has a limited number of neighbours, and the algorithm may revisit locations multiple times. Local search algorithms are often incomplete and may become stuck in a part of the search space from which they cannot escape without employing special mechanisms such as diversification steps or restarting the search process.

## 4.5 Local Search

Among others, the A* search algorithm is best suited to applications in which the cost of the route from the starting state to the goal is the primary objective of the search process. However, in many cases, one only needs to find the final state without worrying (too much) about the path length, and the final state has all the information encoded that we need in the first place. In other cases, one may not have a crisp definition of a goal state, and one only wants to find a state with the least loss (i.e., the objective function), which needs to be minimised. This can occur in settings where the state space is too large to discover a specific goal state, and the state with the least loss is sufficient for the application at hand. In problems such as eight-queens or eight-puzzle settings, there is one or more desired goal states, and heuristic loss values can be defined to measure closeness to these states. Loss functions are defined such that the goal states have zero loss. In other applications, one does not have the notion of a goal state at all but attempts to simply find the state with the least possible loss. Optimisation is the process of setting decision variable values in such a way that the set-out objective is the best possible outcome while satisfying the constraints. In the optimised version of NP-hard problems, each solution can be viewed as a state with an objective function that one is trying to optimise; however, one does not have the notion of a goal state. For example, in the case of the travelling salesperson problem, one would try to find a cycle with the minimum cost. This is extremely common in loss-centric settings, where one tries to optimise an objective function rather than reach a goal. In general, optimisation is a more general problem than that of reaching a goal, because most goal-centric problems can be converted to optimisation-centric problems with the use of appropriately defined loss functions.

## 4.6 Definition of a Local Search

A local search algorithm is a heuristic approach for finding a solution to a problem by iteratively improving a candidate solution. It works by exploring the **"neighbourhood"** of the current solution, looking for better solutions nearby. Think of it like trying to find the highest point in a mountain range by starting at a random spot and repeatedly taking steps uphill.

The local search works as follows:

1. **Start with a candidate solution**- This could be a random solution or one generated by another method.

2. **Define a ”neighbourhood”**- This is the set of solutions that are ”close” to the current solution. The definition of ”close” depends on the specific problem.

3. **Explore the neighbourhood-** Examine the solutions in the neighbourhood of the current solution.

4. **Choose a better neighbour**- If a better solution (according to some evaluation function) is found in the neighbourhood, replace the current solution with that neighbour.

5. **Repeat -** Continue steps 3 and 4 until a satisfactory solution is found or a predefined stopping criterion is met (e.g., a maximum number of iterations, no improvement found in the neighbourhood).

**Algorithm 1** Local Search

**Require:** A problem with initial state and objective function $f(x)$
**Ensure:** A local maximum state

```
current ← MAKE-NODE(INITIAL-STATE)
loop
    neighbour ← highest-valued successor of current
    if f(neighbour) ≤ f(current) then
        return STATE(current) {Peak reached}
    end if
    current ← neighbour
end loop=0
```

## 4.7 Characteristics of Local Search

The biggest hindrance associated with local search, such as hill-climbing, is that of local optima. A local maximum (or minimum) is a peak state in the landscape which is better than each of its neighbouring states, but there is another state also present that is higher than the local maximum. This is shown in Figure 28.

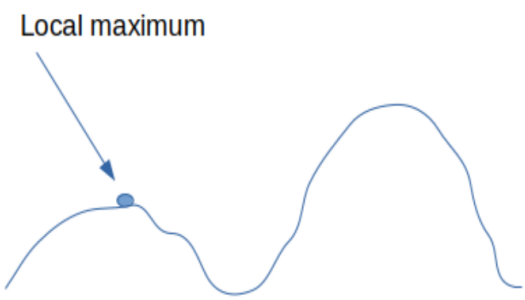


Figure 28: Local Search

The solution to this problem is the backtracking technique in the state-space landscape. Create a list of promising paths so that the algorithm can backtrack the search space and explore other paths, as was done for greedy hill climbing with backtracking. Despite the backtracking algorithm, a hill-climbing search might get lost in a plateau area. A plateau is a flat area of the search space in which all the neighbour states of the current state contain the same value; because of this, the algorithm does not find any best direction to move. Local optima are particularly common in NP-hard problems, such as the travelling salesperson problem. In fact, in almost all attempts to perform hill climbing with NP-hard problems, it is common to get stuck in the local optima. It is difficult to find optimal solutions

to such problems, and it would be too simplistic to expect a simple algorithm, such as hill climbing, to find an optimal solution. The local optimum is the final solution for the hill-climbing search. For example, if one were to examine the decision version of the eight-queens problem in which we want to find a state with zero loss (i.e., no queens attacking one another), a local optimum provides a dead end containing a position where two queens attack one another, and one cannot reach the desired goal with a single transition. It is indeed possible to create an algorithmic alternative to backtracking in the context of the hill-climbing family of solutions. A deterministic approach to achieve this goal is referred to as tabu search( soon to be explored). The following are key characteristics of a local search:

1. **Iterative improvement** -local search repeatedly refines the current solution.
2. **Neighbourhood exploration** -It focuses on exploring solutions that are similar to the current one.
3. **Heuristic** - It doesn't guarantee to find the absolute best solution (global optimum), but it aims to find a good solution (local optimum) in a reasonable amount of time.
4. **Problem-specific** The definition of **"neighbourhood"** and the evaluation function are specific to the problem being solved.

A problem, combined with a way of defining "nearby" solutions (a neighbourhood structure), creates what's called a search landscape. Imagine this landscape as a map where each point is a possible solution, and the height of the land represents how good that solution is. The connections between points show which solutions are neighbours. The absolute lowest point on this map is the globally best solution. Local search methods try to find this lowest point.

The simplest local search, called iterative improvement, only accepts "moves" to neighbouring solutions that are better than the current one. It stops when it can't find any better neighbours, meaning it's found a local minimum (a low point, but maybe not the absolute lowest).

There are two main ways to choose which neighbour to move to.
**"First-improvement"** checks neighbours one by one and takes the better first one.
**"Best-improvement"** checks all neighbours and takes the absolute best one. Using first-improvement in the local search is called first-improvement local search, and using best-improvement is called best-improvement (or steepest descent) local search. Both methods get stuck at local minima, so how well they work depends a lot on how the neighbourhood is defined. Types of local searches,

- Hill climbing
- Simulated annealing
- Tabu search

Local search algorithms offer several compelling advantages in optimisation. Most notably, their implementation is often straightforward. Many local search algorithms are conceptually simple, making them relatively easy to understand and translate into code. The ease of implementation makes them accessible to a wider range of users. Furthermore, the simplicity of these algorithms often translates to lower computational overhead, making them a practical choice in situations where resources are limited. Beyond their ease of implementation, local search methods can tackle large and complex problems. Many real-world optimisation challenges involve vast search spaces, rendering an exhaustive search for the global optimum computationally infeasible. In such scenarios, local search is a valuable alternative. By iteratively exploring the neighbourhood of a candidate solution, these algorithms can efficiently navigate through the solution space and converge towards a local optimum. Although this does not guarantee finding the absolute best solution, it often yields a satisfactory result within a reasonable timeframe, making local search a crucial tool for handling computationally demanding problems. Finally, the adaptability of local search algorithms is significant. The fundamental framework for iteratively improving a solution by exploring its neighbourhood can be readily tailored to a diverse array of optimisation problems. The definition of "neighbourhood" and

the specific mechanisms for exploring it can be customised to suit the unique characteristics of the problem at hand. This adaptability allows local searches to be applied across various domains, from scheduling and resource allocation to machine learning and engineering design, making it a versatile and powerful approach in the field of optimisation.

### 4.8 Simulated Annealing

Think of simulated annealing as a more flexible version of hill climbing. While hill climbing always chooses the best immediate improvement, simulated annealing sometimes accepts a worse option. This seemingly counterintuitive approach—allowing the solution quality to temporarily decrease—is what allows simulated annealing to avoid getting stuck on local optima. If you can only ever move to a better solution, like in hill climbing, you'll inevitably get trapped on a "peak" that isn't the highest. Simulated annealing's ability to occasionally go "downhill" is what lets it explore more of the solution landscape and potentially find the true optimum. Simulated annealing gets its name from the way metals are annealed. In that process, metal is heated to a high temperature and then slowly cooled down to form a stable, low-energy crystal structure. This heating and cooling process is similar to how simulated annealing explores the possible solutions to an optimisation problem. The core idea behind simulated annealing—making less-than-ideal choices to escape getting stuck in local optima—is used in other optimisation methods

**Algorithm 2** Simulated Annealing [1]

**Input:** Initial State $s$, Loss function $L(\cdot)$
**Output:** Best solution found

$CURRENT \leftarrow \{s\}$
$BEST \leftarrow \{s\}$
Set initial temperature $T = T_0$ (magnitude similar to typical loss differences)
$t \leftarrow 1$
**repeat**
  $NEXT \leftarrow$ randomly chosen adjacent state to $CURRENT$
  $\Delta L \leftarrow L(NEXT) - L(CURRENT)$
  **if** $\Delta L < 0$ **then**
    $CURRENT \leftarrow NEXT$
  **else**
    $CURRENT \leftarrow NEXT$ with probability $\min\{1, \exp(-\Delta L/T)\}$
  **end if**
  **if** $L(CURRENT) < L(BEST)$ **then**
    $BEST \leftarrow CURRENT$
  **end if**
  $t \leftarrow t + 1$
  $T \leftarrow T_0/\log(t+1)$ {Example cooling schedule}
**until** no improvement in $BEST$ for $N$ iterations
**return** $BEST$ =0

Simulated annealing begins by setting an initial temperature, $T_0$. This $T_0$ is determined by looking at the typical difference in "loss" between neighbouring solutions. A good starting point for $T_0$ is slightly higher than this average loss difference. The algorithm then repeatedly picks a random neighbour of the current solution. If this neighbour is better, it's accepted. If it's worse, it might still be accepted with a probability determined by the current temperature (the "annealing probability"). Because worse solutions are sometimes accepted, the algorithm keeps track of the best solution found so far. With each iteration, the temperature is reduced—in this example, by dividing $T_0$ by *log(t+1)*, where *t* is the iteration number. There are many ways to reduce the temperature (different "cooling schedules"), each with trade-offs between quickly finding good solutions and avoiding getting stuck in local optima. The probabilistic nature of simulated annealing is similar to how backtracking is used in

search algorithms to escape undesirable states, but instead of systematically going back, it randomly jumps out.

### 4.8.1 Disadvantages of Simulated Annealing

Simulated Annealing's performance depends heavily on getting its settings right, particularly the starting temperature, how quickly the temperature cools down, and how it explores neighbouring solutions. **Finding the best settings** often takes a lot of trial and error and can be time-consuming, as there's no single best approach that works for every problem. Because it explores many possibilities, especially in complex problems, Simulated Annealing can also be **computationally expensive**. It's **also a heuristic**, meaning it doesn't guarantee to find the absolute best solution; it might get stuck on a good, but not perfect, solution. While it's less prone to this than some other methods, it's still possible. Furthermore, it can be **slow** to find a good solution, particularly if it cools down too quickly. While not usually a major issue, for extremely large problems, the **memory required** to run Simulated Annealing could become a concern.

## 4.9 Tabu Search

Tabu search algorithm is quite similar to hill climbing, except that it allows solutions to worsen when they reach local optima or are "almost" A state is considered to be an "almost" local optimum if the only states that improve the objective function were visited earlier.

**Algorithm 3** Tabu Search

```
Set s = s_0. Initial candidate solution
Set length(L) = z. Maximum tabu list length
Set L = {}. Initialise tabu list
repeat
  Generate a random neighbour s'
  if s' ∉ L then
    if length(L) > z then
      Remove the oldest solution from L. FIFO queue
    end if
    Set s' ∈ L
  end if
  if s' < s then
    s = s'
  end if
until (stopping criteria satisfied)
return s =0
```

### 4.9.1 Functionality

Algorithm 3 starts with an initial guess at a solution ($s_0$). It maintains a "tabu list" (L) of recently visited solutions to avoid repeating them and getting stuck. The tabu list has a maximum size (z). The algorithm repeatedly picks a random "neighbour" ($s^{'}$) of the current solution (s). If this neighbour is not on the tabu list, the algorithm checks if the tabu list is full. If it is full, the oldest solution is removed (FIFO – First In, First Out). Then the new neighbour (s') is added to the tabu list. Next, it checks if the new neighbour (s') is better than the current solution (s). If it is, the current solution is updated (s = s'). This process continues until a stopping condition is met (e.g., a certain number of iterations, a time limit, or no improvement for a while). Finally, the algorithm returns the best solution it found. In essence, it explores the neighbourhood, avoids recently visited solutions (thanks to the tabu list), and keeps track of the best solution found so far.

Unlike hill climbing, tabu search does not consistently enhance the objective function. The method's name stems from its use of "taboo" states, which are avoided during the search process. When encountering these states, tabu search randomly selects an alternative state, even if it potentially worsens the objective function value. A taboo data structure list, such as a Hash Table, stores these taboo states and grows as more states are explored. This list is comparable to the visited nodes list in other generic search algorithms. In most practical applications, the list's continuous growth necessitates periodic pruning to maintain a manageable size. The structure list serves to record previously visited states, preventing revisitation. Upon reaching a node, the algorithm checks for unvisited nodes that could improve the objective function value. If found, the algorithm moves to that node. Otherwise, it checks for unvisited neighbours of the current node. If present, a random unvisited node is selected for the next move. The algorithm terminates if no unvisited nodes exist in the current node's neighbourhood, returning the current node as the result. While this node may be a local optimum, it typically outperforms the outcome of a hill-climbing algorithm. In practice, two modifications are commonly employed to enhance performance.

- Instead of selecting any neighbour that can improve the cost or objective function, the best possible unvisited neighbour is commonly used, as defined by the cost function. This will lead to quicker improvements in each step; however, the implementation of each step is slower because of the need to evaluate all unvisited neighbours.
- Instead of allowing the list to grow in an unchecked manner, a bound on the number of nodes in the list is maintained both for computational and space efficiency. When this bound is reached, the node that was added to the list, and the earliest was deleted. The node that was visited the earliest is deleted, because it is more likely to be sub-optimal in terms of its cost (because the cost value generally improves over time). This also makes them less likely to be visited.
- The use of a list that allows deletions is akin to the use of short-term memory. Tabu search also allows various ways of keeping intermediate and long-term memories.

Tabu search is a highly parameterised technique that influences where the search begins in the search space. Therefore, its effectiveness depends on the initial configuration. Additionally, the use of memory limits the search in an environment that is memory-constrained. The stopping criteria have to be carefully selected as the search relies on randomness; it may go on and on.

### 4.9.2 Disadvantages of Tabu Search

- **Can get stuck in local optima -** Local search algorithms can get trapped in local optima, which are solutions that are better than their neighbours but not the best overall.
- **No guarantee of optimality** Local search does not guarantee finding the global optimum.
- **Performance depends on parameters** The performance of local search can depend on the choice of parameters, such as the neighbourhood size and the stopping criterion.

# 5 Game and Game Playing

## 5.1 Search In-Game Playing

One of the earliest areas of artificial intelligence was game playing. In this section, we examine search algorithms that can look ahead to decide which move should be made in a two-person zero-sum game, that is, a game in which there can be at most one winner and two players. First, we will look at games in which the state space is sufficiently small for the entire space to be generated. We then examine examples in which the entire state space cannot be generated. In the latter case, heuristics were used to guide the search. Many board games, such as chess and Go, have an extremely high level of complexity, and the problem can be seen as an attempt to find the best choices of moves from a tree of possibilities corresponding to the board positions obtained by making successive moves. The children of each node correspond to the board positions that are reached using individual moves. These board positions are the states encountered by the gameplaying agent.

## 5.2 Minimax Search Algorithm

If the game tree is sufficiently small, we can generate the entire tree and look ahead to decide which move to make. In a game in which there are two players, and only one can win, we want to maximise the score of one player, MAX and minimise the score of the other MIN. The players in the game are referred to as MAX and MIN players. MAX represents the person trying to win the game and maximise their score. MIN represents the opponent who tries to minimise MAX's score. This technique assumes that the opponent has and uses the same knowledge of the state space as MAX. Each level in the search space contains a label MAX or MIN, indicating whose move is at that particular point in the game. This algorithm is used to look ahead and decide which move to make first. It is mainly used in decision-making and game theory. If the game space is sufficiently small, the entire space can be generated, and leaf nodes can be allocated a win (1), draw (0), or loss (-1) value. These values can then be propagated back up the tree to determine which node to use. In propagating the values back up the tree, a MAX node is given the maximum value for all its children, and the MIN node is given the minimum value for all its children.

**Algorithm 3. Minimax Algorithm**

**Repeat**

1. If the limit of search has been reached, compute the static value of the current position relative to the appropriate player. Report the result.
2. Otherwise, if the level is a minimising level, use the minimax on the children of the current position. Report the minimum value of the results.
3. Otherwise, if the level is a maximising level, use the minimax on the children of the current position. Report the maximum of the results.

**Until the entire tree is traversed**

The minimax algorithm is used to minimise the possible loss for a maximum-loss scenario. The algorithm assumes that both players will play to their best possible ability. At each possible position, the opponent makes the most optimal move. This algorithm is a recursive algorithm for choosing the next move in the game. A decision tree is built, where each layer belongs to either the Max or the Min player. A value is associated with each state or position in the game. The number of branches in the tree represents the possible moves in the game. A move may result in negative, zero, or positive values, depending on the game. As stated for small games, a complete tree can be generated from the beginning e.g Tic-tac-toe, there are 9! (9x8x7. . . . . x1) possible moves at the start of the game, which can be represented. Chess has $10^{120}$ possible moves, which would be difficult to represent.

Figure 29 shows a game tree that represents a two-player game between player Max and player Min. Each layer aka as a ply, represents the state of the game at a particular point. Leaf (terminal)

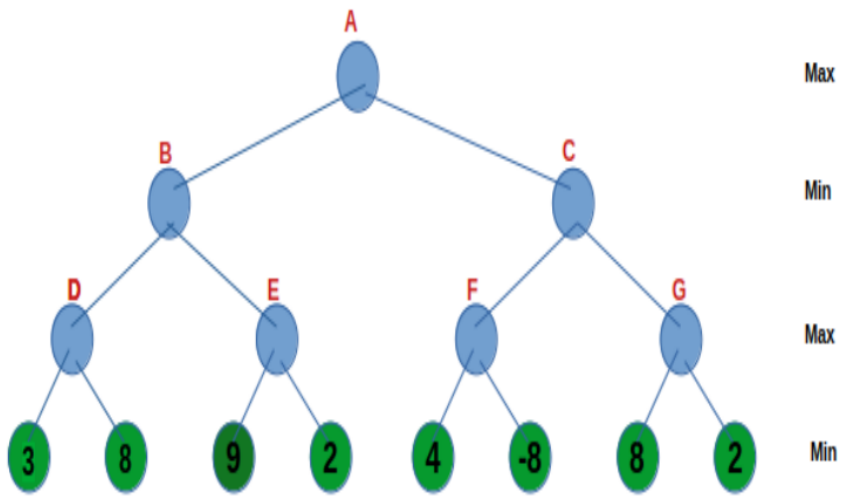


Figure 29:

nodes have values associated with the benefit of taking certain paths. These are known as utility values. Max intends to maximise the utility, and Min intends to minimise. The algorithm applies DFS; therefore, in this game tree, we must go all the way through the leaves to reach the terminal nodes. At the terminal nodes, the terminal values are given, so we compare those values and backtrack the tree until the initial state occurs. The algorithm searches the whole tree using DFS to evaluate the best moves Max(or Min, depending on who has the right to move) can take to make the best decision.

1. **Firstly**
   The algorithm generates the tree and obtains the leaf or utility values (depending on the game). Assuming A is the initial state, and player Max is playing first. Player Max assumes the worst-case scenario of initial values -infinity $(-\infty)$; similarly, Min assumes +infinity$(\infty)$.

2. **Next**
   Depth-first search will be applied from A through B to D. Node D is player Max, and value 3 is obtained from the left child of D. The maximization process max(3, $-\infty$) will be applied because its maximization operation will result in 3 being assigned to D at that stage as 3 is the larger of the two numbers. It will backtrack to D, obtain the value of the other successor node 8, and apply the max() function, that is, max(8,3). Therefore, node D was assigned a value of 8. The algorithm then backtracks to B, which is player Min and the minimisation process B = min(8, $\infty$) = 8. The value 8 is obtained from node D. The process moves to node E, which is player Max, and a similar process that occurred to node D is carried out i.e **E = max(9,$-\infty$)$\rightarrow$ max(9,2) = 9.**

   Figure 30 illustrates the states after this process is complete.

3. **Next**
   In this step, it is the turn of Min to play. Keep in mind the initialised value of **B** for the Min player is $+\infty$, and we are using Depth First search, and at the previous step we had provisionally assigned a value of **8** for **B**. Therefore, effectively, the following occurs;

   - B = min(8,+$\infty$)$\rightarrow$ min(8,9) = 8

   **Node B** is assigned the value 8. The algorithm backtracks to the right branch of **node A** following the **Node B** is assigned the value **8**. The algorithm backtracks to the right branch of **node A**, following the **DFS** approach through **C**, and starts by player Max updating the value of **node F** followed by **G** and **C** as follows: approach through **C** and starts by player Max updating the value of **node F** followed by **G** and **C** as follows:

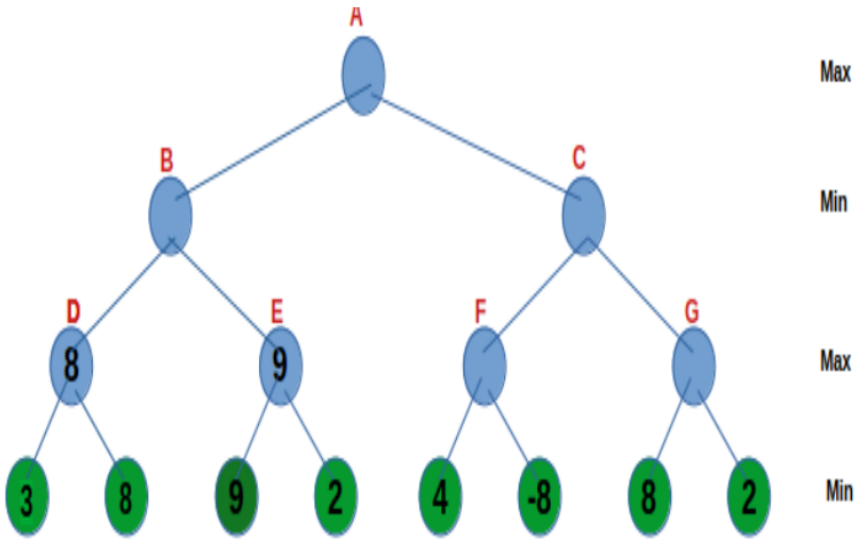


Figure 30:

- F = max(4,-$\infty$)$\rightarrow$ max(-8,4) = 4
- G = max(8,-$\infty$)$\rightarrow$ max(2,8) = 8
- C = min(4,+$\infty$)$\rightarrow$ min(8,4) = 4 (note player Min). Illustrated in Figure 4.

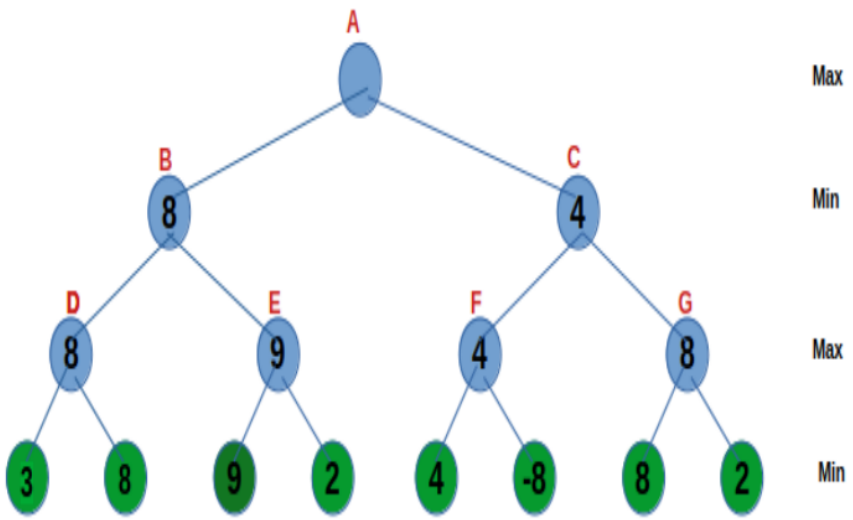


Figure 31:

4. **Next** -Illustrated in Figure 31
   Max plays next. This time, it's a simple move as we are at the initial node.

   - A = max(4,$-\infty$)$\rightarrow$max(8,4) = 8

   This result indicates that to get the most beneficial result, player Max must opt to go to **node B**. An analysis of the utility values along branch B shows that there is a utility value of 9, which is greater than 8; however, Min prevents that from being accessed by Max.

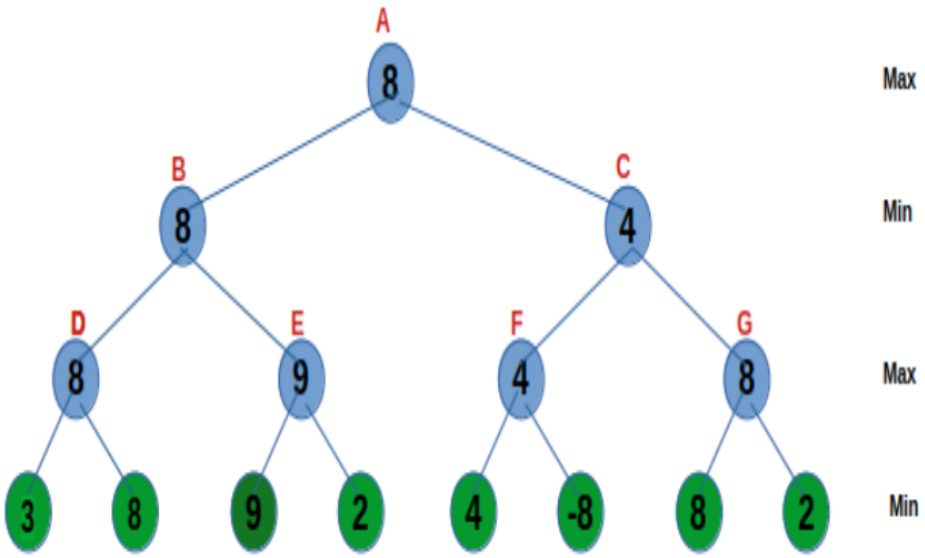


Figure 32:

### 5.3 Characteristics of Minimax

The Minimax algorithm always finds the optimal path in a finite tree, especially if both players play optimally. However, in games with a higher branching factor, the algorithm struggles. If the game space is large, a game tree can be generated at a particular depth or ply. However, win and loss values cannot be assigned to nodes at the cut-off depth because these are not final nodes. Instead, a heuristic value is used. The heuristic values are an estimate of how promising the node is in leading to a win for Max. The depth to which a tree is searched depends on time and computer resource limitations. One of the problems associated with generating a state space to a certain depth is the horizon effect. This refers to the problem of a look-ahead to a certain depth that does not detect a promising path and instead leads to a worse situation. Figure 34 illustrates the minimax algorithm applied to a game tree that has been generated at a specific depth. The integer values at the cutoff depth are calculated using a heuristic function. For example, in chess, the count of the number of white vs black pieces remaining in the game can be a heuristic measure.

The function used to calculate the heuristic values at the cutoff depth is problem-dependent and differs from game to game.

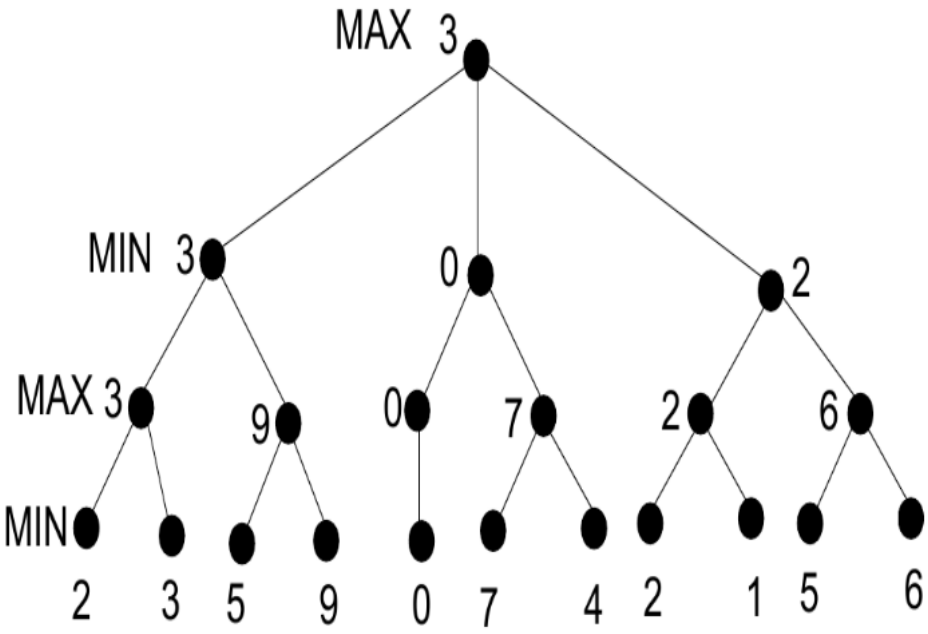


Figure 33:

### 5.4 Alpha Beta Pruning

The main disadvantage of the minimax algorithm is that all nodes in the game tree cut-offs to a certain depth were examined. Alpha-beta pruning reduces the number of nodes explored. Consider the game tree shown in Figure 34. The children of B have static values of 3 and 5, and B is at the MIN level; thus, the value returned to B is three. If we consider F, it has a value of -5. Thus, the value returned to C is at most -5. However, A is at a MAX level; thus, the value at A will be at least three, greater than -5. Therefore, regardless of the value of G, A has a value of 3. Therefore, we do not need to explore and evaluate G because its value will not affect the value returned to A. This is called the alpha cut. Similarly, we can get a beta-cut if the value of a subtree will always be bigger than the value returned by its sibling to a node at a minimising level. In Alpha-Beta pruning, we assign two extra variables to every single node in the search tree: alpha $\alpha$ and beta $\beta$.

- Alpha $\alpha$ stores the best already explored option along a given path to the root of the maximising layer. This is the maximum score that the maximiser can receive; only the maximiser layer can change this value and has an initial value of negative infinity ($-\infty$).
- Beta ($\beta$) stores the best already-explored option along a given path to the root for the minimising layer. This is the minimum score that the minimiser can receive; only the minimiser layer can change this value and has an initial value of positive infinity ($+\infty$).
- IF $\alpha \geq \beta$, a given branch can be pruned, so there is no need to visit the given children.

Algorithm 4 is an outline of the Alpha-Beta Pruning Algorithm.

In Figure 35, in a 2-player game search tree, each $\alpha$ and $\beta$ is set to their initial default values as we descend in a depth-first search approach following the path A, B, D, H. At node H, it is Min's turn to play, and hence the value of $\beta$ is evaluated and updated using the Minimax algorithm, based on the utility values of the nodes, $\beta$ = min($\alpha$,4)$\rightarrow$ min(4,5) = 4. The value of node H is updated to 4, which is the lower value of 4 and 5. Now, the algorithm will backtrack to node D, where the value of $\alpha$ changes as it is the turn of Max. This leads to an update of the of $\alpha$ value to 4 (of $\alpha$= max(-$\infty$,4)), as shown in Figure 36. At this stage, alpha tracks the best case as far as the maximiser is concerned. We can infer that $\alpha$ inherits the $\beta$ value compared to the previous layer. The algorithm then traverses the next successor of **node D**, which is **node I**, and the values of $\alpha$ and $\beta$ will be passed on as shown in Figure 36.

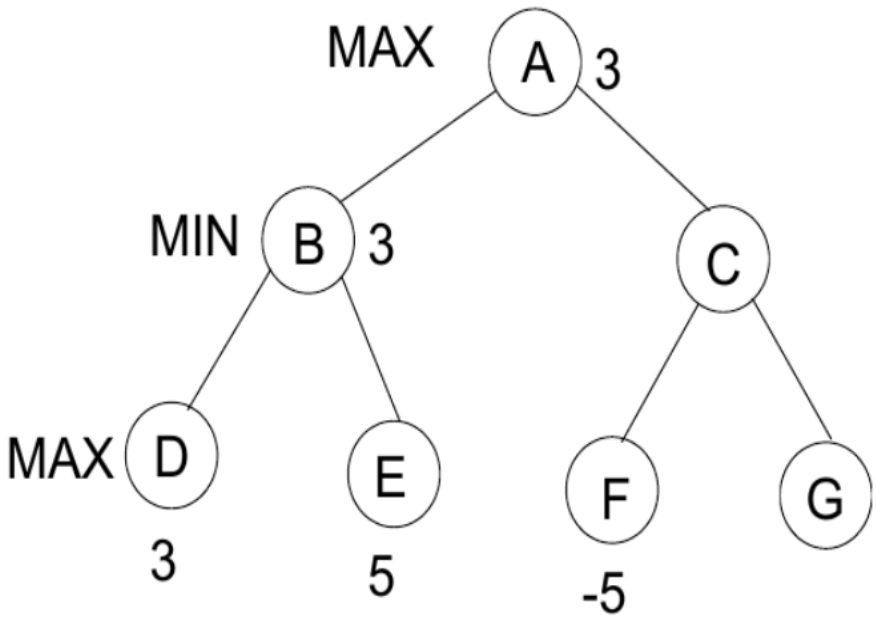


Figure 34:

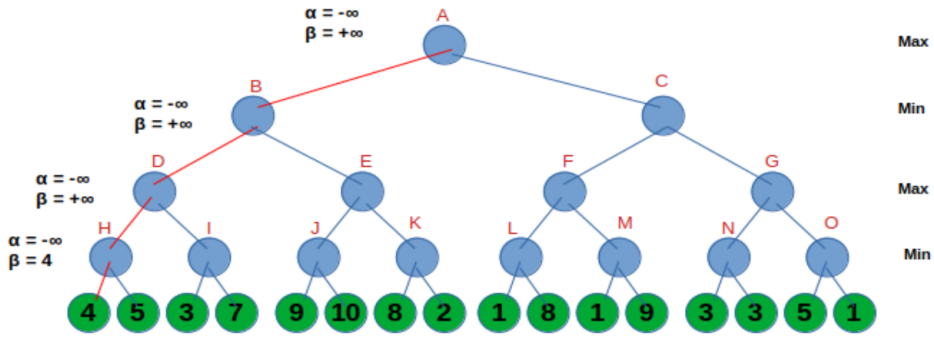


Figure 35:

At node I, Min will take its turn to play, and the value of $\beta$ will change. The current value of $\beta$ will be compared to 3 i.e. $\beta$= min($\infty$, 3) =3. According to the algorithm, if $\alpha$ is greater than or equal to $\beta$ we prune. So the right successor of E will be pruned, and the algorithm will not traverse it. Figure 38 illustrates this process. The value at node I will be 3. Hence at node I $\alpha$= 4 and $\beta$= 3.

The algorithm backtracks to node D, whose value is updated according to the Minimax rules, i.e. max(4,3) = 4. It's important to note that the value of the right successor of node I would have no bearing on this outcome. This is illustrated in Figure 39. The algorithm backtracks to node B, and the $\beta$ value is updated $\beta$ = min(-$\infty$,4) as it is the Min player's turn to play, and the value is set to 4 from the node D. The $\alpha$ and $\beta$ values of node B are passed down to its successor E, which also passes them down to its successor J, following the DFS approach. In Figure 13, the $\beta$ value of node J (Min player) remains 4 as it is less than 9; however, as $\alpha$ is smaller than $\beta$, we explore the other successor path of J, whose value is 10. From the Minimax algorithm, node J is updated with the least of the two values, i.e. 9 and indicated in Figure 40.

The algorithm backtracks to node E, and the $\alpha$ value (Max player) is updated with a value of 9 since 9 is greater than -$\infty$ at that stage, shown in Figure 41. As $\alpha > \beta$ we prune the right branch of the node E and update the value of E to 9. The algorithm backtracks to node B and evaluates the value of B using min(4,9) = 4, as it is the Min player's turn. The algorithm backtracks to node A, where it is the Max player's turn and the $\alpha$ value is evaluated and set to 4 from max($\infty$, 4), and the $\beta$

**Algorithm 4** Minimax Algorithm with Alpha-Beta Pruning

```
0: procedure MINIMAX(node, α, β)
0:   if node is a leaf then
0:     return value(node)
0:   end if
0:   if node is a MAX node then
0:     for each child c of node do
0:       α ← max(α, MINIMAX(c, α, β))
0:       if α ≥ β then {Beta cutoff}
0:         break
0:       end if
0:     end for
0:     return α
0:   else{MIN node}
0:     for each child c of node do
0:       β ← min(β, MINIMAX(c, α, β))
0:       if β ≤ α then {Alpha cutoff}
0:         break
0:       end if
0:     end for
0:     return β
0:   end if
0: end procedure=0
```

value remains $+\infty$. These values are passed to node A's right successor, i.e. C, and the process will continue across the game tree along that path. In summary, the Alpha-Beta pruning process is an optimisation technique for the Minimax algorithm. It cuts off branches in the game tree which need not be searched because there already exists a better move available.

It is called Alpha-Beta pruning because it passes 2 extra parameters in the minimax function, namely alpha and beta. Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prunes the tree leaves but also the entire subtree. The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not affect the final decision, making the algorithm slow. Hence, by pruning these nodes, it makes the algorithm fast.

**Exercise**

Complete the Alpha-beta pruning example, traversing the right side of the game tree to establish which optimal path should be selected by player Max.

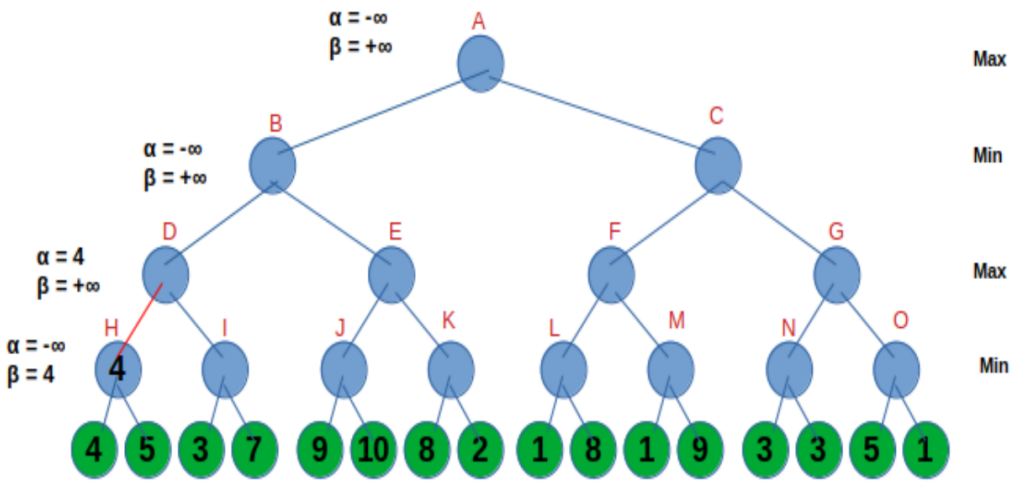


Figure 36:

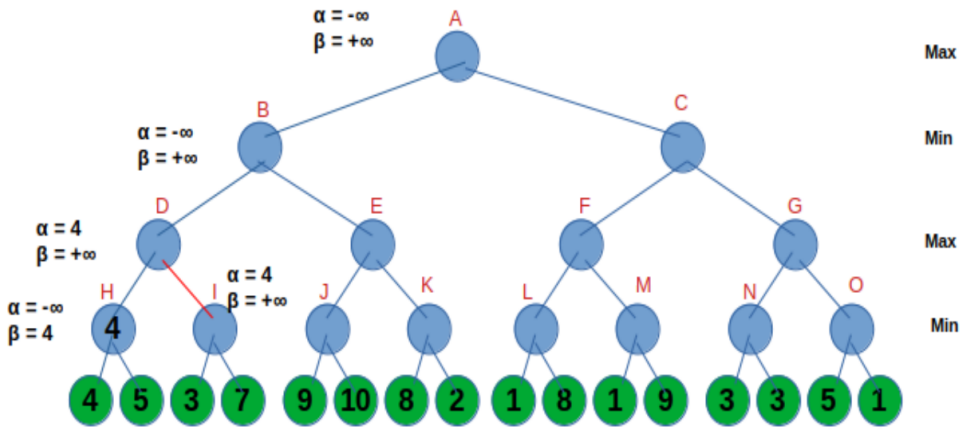


Figure 37:

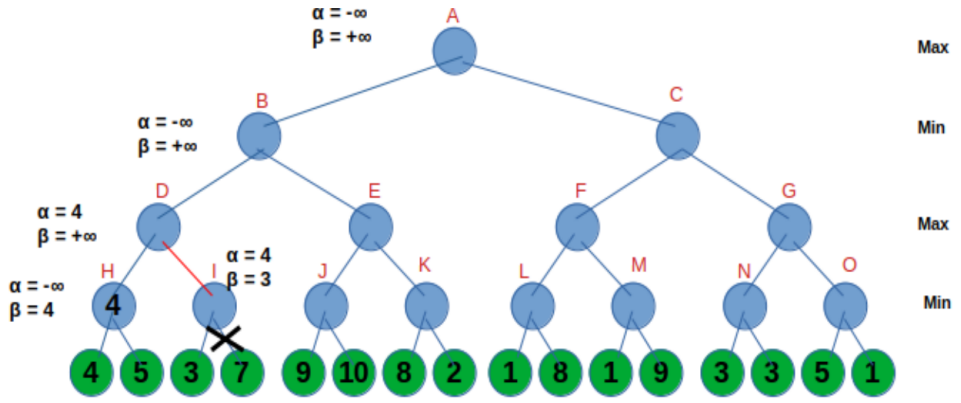


Figure 38:

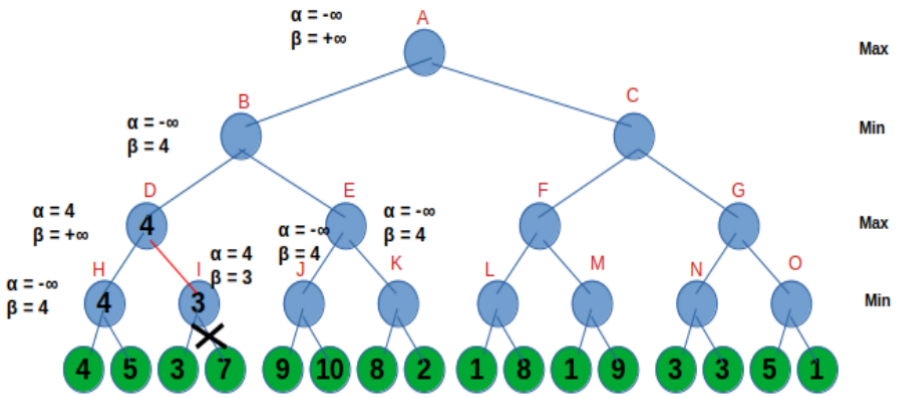


Figure 39:

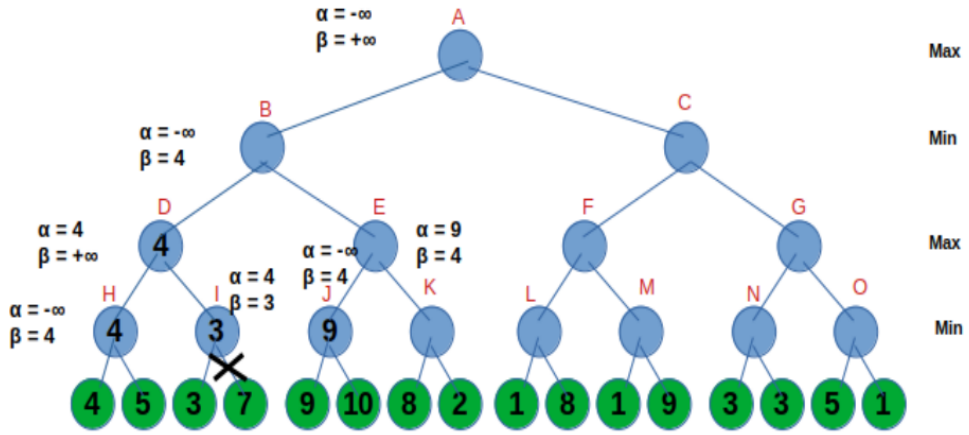


Figure 40:

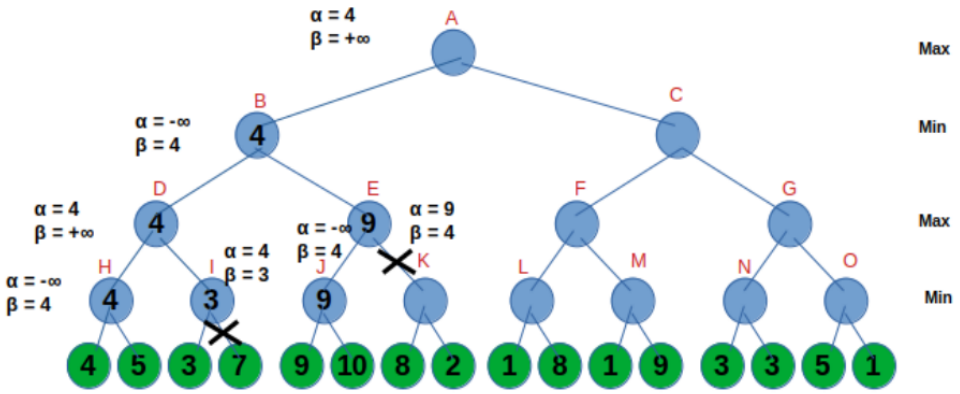


Figure 41:

# 6 Metaheuristics

## 6.1 Introduction

When faced with difficult optimisation problems, particularly those where precise solutions are elusive or computationally expensive, metaheuristics offer a robust and versatile set of problem-solving techniques. These sophisticated algorithmic approaches are designed to explore extensive and complex solution spaces, aiming to identify high-quality, though not necessarily perfect, solutions. The key difference between traditional heuristics and metaheuristics lies in their generality. While heuristics are often tailored to solve a very specific problem, metaheuristics provide a broader, more adaptable framework. They are not confined to a single problem type and can be applied to a wide variety of optimisation challenges. Furthermore, metaheuristics employ strategies to balance the exploration of new potential solutions with the exploitation of promising areas already discovered. This ability to escape local optima, where simpler heuristics might get stuck, is what makes metaheuristics particularly valuable for real-world scenarios that require efficient and effective solutions. "**A metaheuristic is formally defined as an iterative generation process which guides a subordinate heuristic by combining intelligently different concepts for exploring and exploiting the search space, learning strategies are used to structure information to find efficient near-optimal solutions.**"

## 6.2 Metaheuristics Characteristics

The following characterise metaheuristics

- Meta-heuristics algorithms are strategies that determine the path and direction of the search process.
- Metaheuristics are usually stochastic.
- Meta-heuristics have a mechanism that enables them to escape local minima (maxima).
- Meta-heuristics are usually general and non-specific.
- May have domain knowledge in the lower-level heuristics.

## 6.3 Classification of Metaheuristics

Metaheuristics can be classified as single-point or multipoint-based searches. In a single point search, the algorithm works on one solution at a time. In multipoint based search, the search is conducted in parallel on multiple points.

## 6.4 Single Point

These algorithms are viewed as walks through Neighbourhoods in the search space (solution space). The walks are performed via iterative procedures that move from the current solution to the next one. Iterative procedures consist of generation and replacement from a single current solution.
**Generation phase:** A set of candidate solutions $C(s)$ are generated from the current solution $s$ by local transformation.
**Replacement phase:** Selection of $s'$ is performed from $C(s)$ such that the objective function $f(s')$ is better than $f(s)$. The iterative process continues until a stopping criterion is reached. The generation and replacement phases may be memoryless. Otherwise, some history of the search is stored for further generation of candidate solutions.
Key elements: define the Neighbourhood structure and the initial solution. Some examples of single-point metaheuristic searches are *Simulated Annealing, Tabu Search, Iterated Local Search, and Variable Neighbourhood Search*.

### 6.4.1 Iterated Local Search

Iterated Local Search (ILS) is a metaheuristic algorithm designed to improve upon the results of a basic local search procedure. It's particularly effective for optimisation problems where local search methods might get trapped in local optima, preventing them from finding globally optimal or near-optimal solutions. ILS achieves this by iteratively perturbing locally optimal solutions and then applying local search again to explore new areas of the solution space. By strategically introducing diversity and intensification, ILS aims to navigate the search landscape more effectively, leading to better solutions than simple local search alone.

**Algorithm 5** Iterated Local Search (ILS) [1]

0: **1.** $s_1 \leftarrow$ GenerateInitialSolution
0: **2.** $s^* \leftarrow$ LocalSearch$(s_1)$
0: **repeat**
0: **3.** $s' \leftarrow$ Perturbation$(s^*, \text{history})$
0: **4.** $s^{*'} \leftarrow$ LocalSearch$(s')$
0: **5.** $s^* \leftarrow$ AcceptanceCriterion$(s^*, s^{*'}, \text{history})$
0: **until 6.** termination condition met =0

**Step-by-Step Description of the ILS Algorithm**
Here's a detailed breakdown of the ILS algorithm based on the provided pseudocode
**1. Generate Initial Solution ($s_1 \leftarrow$ GenerateInitialSolution)**
The algorithm begins by creating an initial solution ($s_1$). This can be done randomly or using a problem-specific heuristic. The quality of this initial solution can influence the algorithm's performance.
**2. Perform Local Search ($s^* \leftarrow$ LocalSearch$(s_1)$)**
A local search procedure is applied to the initial solution ($s_1$) to find a locally optimal solution ($s^*$). Local search typically involves making small changes to the current solution and accepting those that improve the objective function. This step aims to find a good starting point for the iterative process.
**3. Perturbation ($s' \leftarrow$ Perturbation$(s^*, \text{history})$)**
The algorithm introduces a perturbation (a change) to the current locally optimal solution ($s^*$), resulting in a new solution ($s'$). This perturbation aims to move the search away from the current local optimum. The history parameter might be used to avoid repeating previously explored areas. The strength of the perturbation is a key parameter of the ILS.
**4. Perform Local Search Again ($s^{*'} \leftarrow$ LocalSearch$(s')$)**
Another local search is performed on the perturbed solution ($s'$) to find a new locally optimal solution ($s^{*'}$). This step aims to refine the perturbed solution and find a better local optimum in the new region of the search space.
**5. Acceptance Criterion ($s^* \leftarrow$ AcceptanceCriterion$(s^*, s^{*'}, \text{history})$)**
The algorithm decides whether to accept the new locally optimal solution ($s^{*'}$) or keep the previous one ($s^*$). This decision is based on an acceptance criterion, which might consider the quality of the new solution, the history of the search, or other factors. For example, it might accept a worse solution with a certain probability to avoid getting stuck in local optima. If the new solution is accepted, it replaces the previous best solution.
**6. Termination Condition Check (until termination condition met)**
The algorithm checks if the termination condition has been met. This condition might be a maximum number of iterations, a time limit, or a target solution quality. If the condition is met, the algorithm terminates; otherwise, it returns to step 3 (perturbation) and continues the iterative process.

### 6.4.2 Case Study - Knapsack Problem

*The Knapsack problem is a classic NP-hard optimisation challenge involving selecting a subset of $n$ items, each with weight $w_i$ and value $v_i$, to maximise the total value within a knapsack of capacity $W$. Due to its NP-hard nature, optimal solutions are computationally expensive for large instances.*

*However, efficient algorithms and metaheuristics provide acceptable approximate solutions. The Knapsack problem finds applications in diverse fields, including logistics, finance, scheduling, and resource allocation.*

Given the following instance of the 0/1 Knapsack problem

f3l-dkp4-20= {red{ 4,20},{9,6 },{11,5 },{13,9 },{15,7 } } [2]

- Where red{ 4,20} = ”4” indicates the number of items available. ”20” represents the capacity of the knapsack (the maximum weight it can hold).
- The subsequent lines represent the items. Each line corresponds to one item and contains two numbers: the item’s weight and its value, respectively.
  - ”9 6”: Item 1 has a weight of 9 and a value of 6.
  - ”11 5”: Item 2 has a weight of 11 and a value of 5.
  - ”13 9”: Item 3 has a weight of 13 and a value of 9.
  - ”15 7”: Item 4 has a weight of 15 and a value of 7.

You have a knapsack with a capacity of 20 units of weight. You’re given four items, each with its weight and value. The goal of the knapsack problem in this instance is to find the combination of these four items that you can fit into the knapsack (without exceeding the weight capacity) that maximises the total value of the items inside. Use the Iterated Local Search to solve this problem.

1. **Solution Representation**- you need to establish how you will represent the solution. This is a problem-dependent. Represent a solution as a binary string of length 4.
   - 1 means the item is included.
   - 0 means the item is excluded.

   For example, 1010 means items 1 and 3 are in the knapsack, and items 2 and 4 are not.
2. **Initial Solution generation**. For simplicity, you can start with a random solution. E.g. 1100 (items 1 and 2).
   - red**Initial Solution: 1100 (weight: 9 + 11 = 20, value: 6 + 5 = 11)**
3. **Local Search**.
   - Define a Neighbourhood: A Neighbour is created by flipping one bit (adding or removing an item).
   - Local Search Procedure.
     - Generate all Neighbours of the current solution.
     - Evaluate each Neighbour: Calculate the total weight and value. If the weight is within the capacity (20) and the value is higher than the current solution, accept the Neighbour as the new current solution.
     - Repeat until no better Neighbour is found.
   - Repair Function: If a Neighbour exceeds the weight capacity, it is invalid. In this simple example, we can just reject the Neighbour. For more complex problems, more sophisticated repair functions are necessary.
     - **red**

       **Neighbours: 0100, 1000, 1110, 1101.**
     - **1110 (weight: 9 + 11 + 13 = 33) is invalid.**
     - **1101 (weight: 9 + 11 + 15 = 35) is invalid.**
     - **0100 (weight: 11, value: 5) is lower.**

  - **1000 (weight: 9, value: 6) is lower.**
  - **So, 1100 remains.**

4. **Perturbation**

   - Introduce a perturbation, e.g. flip two bits randomly. This will change the solution significantly, moving it to a different area of the search space.
   - Example: If the current solution is 1010, a perturbation might flip bits 1 and 4, resulting in 0011.

     **redNew perturbation of 0101 (weight 11+15=26) change 2 bits invalid, therefore, we try again.**
     **New perturbation of 0010 (weight 13, value 9)**

5. **Local Search Again**

   - Apply the local search procedure (step 3) to the perturbed solution.
     **redNeighbours 1010 = invalid.**
     **Neighbours 0110 = invalid**
     **Neighbours 0000 = value 0**
     **Neighbours 0011 = invalid.**
     **0010 remains.**

6. **Acceptance Criterion**

   - Simple acceptance: If the new locally optimal solution (s*') has a higher value than the current best solution (s*), accept it.
   - You could use a probabilistic acceptance criterion (like in simulated annealing) to occasionally accept worse solutions, helping to escape local optima.

   **red0010 has value of 9, and 1100 has value of 11. 1100 is kept.**

7. **Termination Condition**

   - Set a maximum number of iterations (e.g., 100 iterations). The algorithm stops when the termination condition is met.
     **redRepeat: Continue with perturbation, local search, and acceptance.**

# 7 Multipoint Search

Multipoint searches, a fundamental concept in optimisation and search algorithms, deviate from traditional single-point methods by simultaneously exploring multiple regions of the solution space. This approach leverages a population of candidate solutions, allowing for the concurrent evaluation and manipulation of diverse search pathways. By maintaining and evolving a collection of potential solutions, multipoint searches aim to mitigate the risk of premature convergence to local optima and foster a more robust exploration of the search landscape. This strategy is particularly effective in complex, high-dimensional problems where the solution space is vast and potentially contains multiple promising areas, enabling a more efficient and thorough search for optimal or near-optimal solutions. In this module, we will consider evolutionary computation for this section of the module

Evolutionary computation is a subfield of artificial intelligence that models the Darwinian principle of natural selection as inspiration for the design of computational methods. Algorithms that follow the principle of natural selection are generally referred to as Evolutionary Algorithms (EAs). They operate on a collection of candidate solutions for a given problem. Each candidate solution is represented as an individual, and a collection of individuals is called a population. Each individual in the population is assigned a quality value known as a fitness, which is evaluated by a function called a fitness function. The fitness function measures how well a candidate solution solves the problem being considered.

Evolutionary algorithms are iterative, and each iteration is called a generation. Algorithm 2 presents the pseudocode of a generic EA.

**Algorithm 6** Generic Evolutionary Algorithm (EA)

```
Initialise population with random individuals
Define fitness function to evaluate individuals
while termination condition not met do
    Evaluate fitness of each individual
    Select individuals for reproduction based on fitness
    Apply genetic operators (crossover and mutation) to create offspring
    Replace the old population with new offspring
end while
Return the best solution found =0
```

Based on Algorithm 7, an EA generally operates as follows: as a first step, a **population** of individuals is **initialised** before evolution can take place. The initial population is typically generated **randomly**. The next step is to **evaluate** the fitness of each individual using a **fitness function**. A predefined termination condition is then checked. If the termination condition is met, the algorithm outputs the best solution for the problem and terminates. If the **termination condition** is not met, a selection method is used to choose individuals from the population to undergo **genetic manipulation**. The selection method tends to be biased towards fitter individuals. The selected individuals are used to seed the next generation of individuals. This is achieved through the application of **genetic operators (crossover and/or mutation)** to the selected individuals (parents). The fitness of the resulting individual(s) (offspring) is evaluated. A population replacement strategy is then used to update the population. This process continues iteratively until a **stopping criterion** is met.

# 8 Genetic Algorithm

Individuals in a Genetic Algorithm [3] population represent possible solutions to the problem being solved. An individual is typically encoded as a fixed-length linear chromosome, where each gene of the chromosome is a binary bit string. However, other types of encoding can be used, such as real numbers or alphabet characters. The first step of a basic genetic algorithm is to randomly create an initial population of individuals. The fitness of each individual in the population is then evaluated. A selection method is used to select individuals from the current population to act as parents and undergo crossover and mutation to create offspring for the next generation. The population is then updated. This process continues iteratively until a predefined stopping criterion is met, and the algorithm terminates. The best solution, based on fitness, is returned. Algorithm 8 outlines the steps of a typical GA algorithm.

**Algorithm 7** Genetic Algorithm (GA)[3]

```
Initialise population with random individuals
Define fitness function to evaluate individuals
while termination condition not met do
    Evaluate fitness of each individual
    Select individuals for reproduction based on fitness
    Apply genetic operators (crossover and mutation) to create offspring
    Replace the old population with new offspring
end while
Return the best solution found =0
```

## 8.1 Initial Population Generation

During the initialisation step, individuals are created randomly. The value of each gene is randomly selected from a range of possible values for that gene, as determined by the encoding scheme. The number of chromosomes created (the population size) is determined by a population size parameter. The encoding used depends on the problem being solved, as each individual in a Genetic Algorithm (GA) represents a candidate solution.

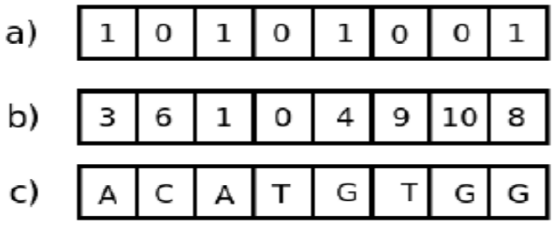


Figure 42: GA: Encoding Schemes

Figure 42 shows three examples of typical Genetic Algorithm (GA) individuals. Each individual exemplifies a specific encoding scheme. Example a) illustrates the binary encoding scheme, where each gene can have a value of either 1 or 0. Example b) is for the real number encoding scheme, and example c) shows an individual encoded using characters from the alphabet. The range of possible values for each gene is specified before initial population generation to ensure each randomly created individual is a valid candidate solution for the problem being solved. For example, to encode the Travelling Salesman Problem (TSP), a chromosome (individual) can represent a tour (solution) of the cities.

## 8.2 Fitness Evaluation

Fitness is a measure of the quality of each individual's ability to solve the problem at hand. A fitness function is used to evaluate the fitness of an individual. The fitness function is typically predefined. It can be a mathematical function or a measure of how well a Genetic Algorithm (GA) individual solves the specified problem.

## 8.3 Selection

A selection method is used to select individuals from a population. Tournament selection, an example of a selection method, follows a simple approach. A fixed number of individuals, t, are randomly selected from the population, and the individual with the best fitness from the t individuals is returned.

## 8.4 Crossover

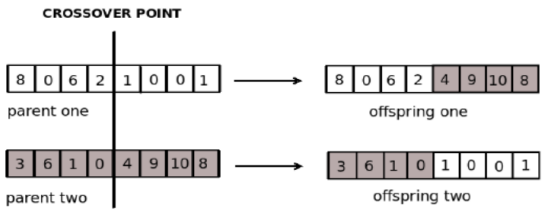


Figure 43: Single point Crossover

Crossover is considered the primary genetic operator for a GA. In crossover, usually two individuals are selected using a selection method to act as parents, which exchange elements of their chromosomes before and after randomly selected crossover point(s) to create one or two offspring. The likelihood of crossover occurring is determined by a crossover probability, pc. A random number, r, is chosen between [0, 1], and if r is less than pc, then crossover takes place. The objective of a GA is to achieve convergence; therefore, crossover is usually applied at higher rates. There are several crossover types. Figure 43 is an illustration of a single-point crossover.

### 8.4.1 Mutation

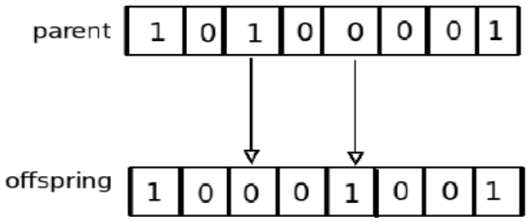


Figure 44: Flip Bit Mutation

Mutation, illustrated in Figure 44, is applied to a single individual after the individual has resulted from crossover. Mutation is used to provide diversity within a population and involves making small, random changes in the chromosome, resulting in a new individual. Mutation plays an important role in preventing the search from prematurely converging to a local minimum/maximum. The application rate of mutation, given by $p_m$, is normally less than the crossover rate. A very high mutation rate may result in the search becoming a random search.

### 8.4.2 Population Update

The two commonly used population replacement methods for Genetic Algorithms (GAs) are generational and steady-state. In generational replacement, the current population is entirely replaced by a new population generated from crossover and mutation. Steady-state replacement replaces a specific number, n, of individuals in the current population with n offspring.

### 8.4.3 Termination

The algorithm evolves from generation to generation until a termination criterion is met. Genetic algorithms are stochastic, and there is no guarantee that the algorithm will either converge or find the optimal solution. The termination criterion can be set to a maximum number of generations or a near-optimal solution. The complete execution of a genetic algorithm, from population initialisation to executing the specified number of generations and outputting the solution, is known as a run.

## 9 Genetic Programming

Genetic programming (GP) is an Evolutionary Algorithm (EA) that explores a program space. Proposed by Koza, GP is considered an extension of genetic algorithms. In GP, individuals are computer programs, and the aim is to evolve fitter programs through population evolution until a program providing a near-optimal solution is generated.

The basic approach of a GP algorithm involves initially creating a population of randomly generated programs. Each program is constructed from building blocks necessary to solve the problem GP is applied to. The fitness of each randomly generated program is then evaluated. If a specified

termination criterion is not met, suitable programs are selected as parents for generating new programs. New programs are created by applying genetic operators to these parent programs, and their fitness is evaluated. This process of selecting suitable programs and applying genetic operators is repeated until a stopping criterion is met, and the best program is output. Therefore, unlike a Genetic Algorithm (GA), which searches for a solution in a solution space, GP searches for a program in a program space to solve a given problem.

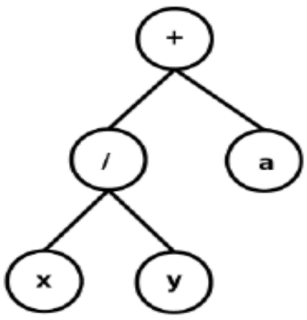


Figure 45: GP Individual

Figure 45 provides an example of a syntax tree whose executable expression is as follows:

$$f = (x/y) + a \tag{4}$$

In GP, programs (individuals) are traditionally represented as syntax trees, which can be converted to their corresponding executable expressions, typically in prefix notation. Each node of the tree is considered a gene. The internal nodes of a syntax tree are known as functions, while the external nodes are known as terminals. Functions and terminals are constructed from elements of a function set and a terminal set, respectively. A function set comprises application-specific operators, and a terminal set comprises inputs to the GP program for the problem being addressed. Function and terminal sets are problem-dependent primitives from which GP programs are built.

The operators in a function set are defined with an arity, which represents the number of inputs a specific operator requires. The function set may include:

- arithmetic operators {+, -, *, /};
- mathematical functions {sine, cosine, sqrt};
- logical operators {AND, OR, NOR, NOT};
- equality operators {<,>, <=,>=,==};
- user-defined domain-specific operators.

Two important properties that a function set and terminal set must satisfy are the **closure property** and the **sufficiency property**. The closure property requires that any output from any function or terminal must be a valid input for all other functions. To meet this property, the functionality of some operators is modified. A common example is the arithmetic division operator, whose functionality is modified to enable division by zero. Typically, it returns a 1 or 0 when a division by zero occurs. This is known as a protected division. The sufficiency property requires that elements of the function and terminal sets be capable of representing the solution.

## 9.1 GP Algorithm

**Algorithm 8** Genetic Programming (GP) [4]

```
Initialise population with random individuals (programs)
Define fitness function to evaluate individuals
while termination condition not met do
    Evaluate fitness of each individual
    Select individuals for reproduction based on fitness
    Apply genetic operators (crossover and mutation) to create offspring
    Replace the old population with new offspring
end while
Return the best solution found =0
```

### 9.1.1 Initial Population Generation

Like most Evolutionary Algorithms (EAs), the initial population generation of GP programs is performed randomly. The number of programs generated is specified by a user-defined population size parameter. To create a program, a function for the root node is randomly selected from the function set, and the rest of the tree is recursively constructed, with the leaves of any new nodes filled until the maximum tree depth is reached. Maximum tree depth is defined as the number of nodes between an end node of a tree and the root node. The value of this parameter is user-defined. Three methods exist for initial tree generation, namely, **full, grow** and **ramped half-and-half**. The full method constructs trees in such a manner that all the nodes up to a depth of (maximum tree depth - 1) are functions, and a depth equal to the maximum tree depth are terminals. The grow method creates trees of variable length. Nodes between the root and (maximum tree depth - 1 ) may randomly be assigned as functions or terminals, and those at the maximum tree depth are set as terminals. The **ramped half-and-half** method combines the full and grow method.

### 9.1.2 Fitness Function

The effectiveness of a Genetic Programming (GP) program is measured using a fitness function, which is typically problem-dependent. Each program is applied to a training set (fitness cases), and the result is assigned as the fitness of the program. Fitness can be measured in several ways. For example, it can be a measure of how close a program's output is to a desired outcome, such as the accuracy rate in classification, or a measure of how quickly a GP program provides a solution.

### 9.1.3 Selection

As in other evolutionary algorithms, selection in GP is biased towards fitter individuals. The two commonly used selection methods are tournament selection and fitness-proportionate selection.

### 9.1.4 Genetic Operators

Crossover, mutation, and reproduction are the commonly used genetic operators. In crossover, two programs selected as parents exchange code to create two new programs, while in mutation, a random change is made to a selected parent program.

- **Subtree Crossover**

  An exchange of subtree branches between parents. Two parents are selected from the population using a selection method. A random crossover point is selected on each parent. Subtrees rooted at the randomly selected crossover points are known as crossover fragments. The two fragments are exchanged from one parent to the other, thus creating two offspring. The size of the offspring must not exceed a user-defined offspring depth limit. Those that do must be pruned. This is shown in Figure 46.

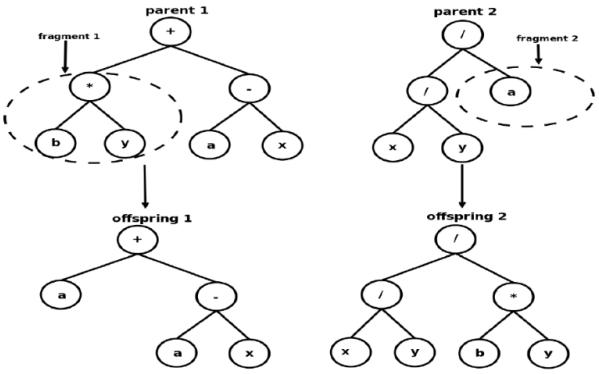


Figure 46: Sub Tree Crossover

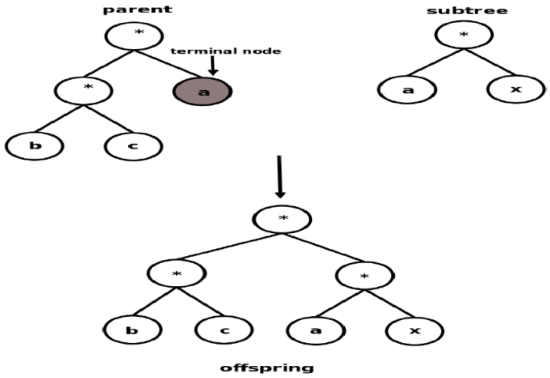


Figure 47: Grow Mutation

- **Mutation**
  Mutation is an important operator for increasing diversity in a population during evolution. The two commonly used mutation types are **Grow** and **Shrink** mutation.

  Shrink mutation is an operator that replaces a randomly selected subtree with a randomly created terminal node. Grow mutation works by randomly selecting a terminal and replacing it with a subtree. Figure 47 illustrates grow mutation. The grow mutation has the effect of increasing the size of a tree. The resultant offspring needs to conform to the specified offspring depth, similar to the offspring resulting from crossover. Therefore, a parameter known as mutation depth is specified for a GP system. The purpose of this parameter is to control the size of a subtree created by a mutation operator. The population size of the new generation is the same size as the initially randomly generated population. Part of the new population is evolved by crossover and the other part by mutation. The number of individuals created by crossover and those created by mutation are determined by the application rate parameters, and they should sum up to the population size. The application rates are known as the crossover rate and the mutation rate.

### 9.1.5 Population Replacement

The most frequently used population replacement methods are generational and steady-state. In a steady state, the population is updated in such a manner that one offspring replaces a member of the current population based on fitness.

### 9.1.6 Termination

Two termination conditions are usually used in GP: a maximum number of generations or a problem-specific solution is met. The maximum number of generations is a user-defined parameter and is specified before a run.

## 9.2 Applications of Genetic Programming

Genetic programming (GP) is a powerful evolutionary computation technique that excels in automating the creation of computer programs. Its application areas span a wide range of fields, primarily where the goal is to discover or optimise a program or model. Here's a summary of its key application areas:

1. **Symbolic Regression**: GP is highly effective in discovering mathematical expressions or models that fit a given dataset. This is crucial in fields like data mining, scientific modelling, and financial analysis.
2. **Automatic Programming:** GP can generate computer programs to solve specific tasks, such as sorting algorithms, control systems, and even game-playing AI.
3. **Machine Learning and Classification**: GP can be used to evolve classification rules, decision trees, and other machine learning models, often demonstrating competitive performance compared to traditional techniques.
4. **Control Systems:** GP is applied to design controllers for robots, autonomous vehicles, and other dynamic systems, optimising their behaviour and performance.
5. **Circuit Design**: GP can evolve electronic circuits and hardware designs, leading to optimised or novel solutions. Image and Signal Processing: GP can develop algorithms for image recognition, signal filtering, and other processing tasks.
6. **Financial Modelling :** GP is used to create predictive models for stock market trends, risk assessment, and other financial applications.
7. **Game Playing**: GP can evolve strategies and programs for playing games, often resulting in complex and adaptive behaviours.
8. **Robotics**: GP is used to create control programs for robots, allowing them to adapt to changing environments and learn new tasks.
9. **Optimisation**: GP can be used to evolve functions that optimise complex systems.

In essence, GP is valuable in any domain where the goal is to automatically generate or optimise a program or model, particularly when the underlying relationships are complex or unknown. Its ability to evolve complex structures makes it a versatile tool for tackling a wide range of challenging problems.

# 10 Grammatical Evolution

## 10.1 Introduction

Grammatical Evolution, proposed by Ryan et al. [6], is considered to be an extension of Genetic Programming (GP). Unlike GP, which uses syntax trees to represent individuals, Grammatical Evolution (GE) uses chromosomes of variable-length binary strings. Each gene of the chromosome is an 8-bit binary string and is referred to as a codon. Codons contain information on how to select production rules from a programming language grammar. Grammatical Evolution uses the context-free Backus-Naur Form grammar. Figure 48 illustrates two examples of GE individuals, where a) is a genome consisting of 6 codons and b) consists of 8 codons.

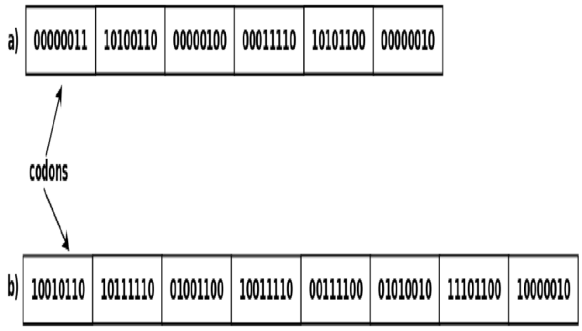


Figure 48: Codons

Grammatical Evolution uses a user-defined Backus-Naur Form (BNF) grammar to map variable-length linear genomes to executable programs. Domain knowledge of the problem being solved is incorporated into the grammar. The grammar is used in conjunction with evolution to map a genotype to a phenotype. Grammatical Evolution draws inspiration from molecular biology, where a sequence of genetic material, deoxyribonucleic acid (DNA) (genotype), is translated into a protein, which defines the characteristics of a phenotype. A grammar G can be represented by the four-tuple <N, T, P, S>, where N represents a set of non-terminals, T a set of terminals, P a set of production rules that map the elements of N to T, and S (a member of N) the start symbol. An individual (genotype) is used to map the start symbol S to terminals by reading and converting each codon to its decimal value, from which an appropriate production rule is selected using the following mapping function:

$$Rule = (codondecimalvalue)\%(Numberofproductionrules) \tag{5}$$

A derivation tree (phenotype) is evolved by iterating and mapping through the sequence of codons. The derivation process is performed from left to right, starting with the leftmost non-terminal. If the iteration process reaches the end of the sequence of codons before the derivation tree is fully evolved, the procedure continues by looping to the start of the codon sequence, a process called wrapping. The fitness of the phenotype is evaluated by applying it to the problem at hand.

## 10.2 GE Algorithm

**Algorithm 9** Grammatical Evolution (GE) [5]

```
Initialise population with random individuals
Define grammar and mapping rules
Evaluate fitness of each individual
while termination condition not met do
    Select individuals for reproduction
    Apply genetic operators (crossover and mutation)
    Generate new population
    Map new individuals to syntactically correct programs using grammar rules
    Evaluate fitness of new individuals
end while
Return the best solution found =0
```

### 10.2.1 Initial Population Generation

The first step of the Grammatical Evolution (GE) algorithm is to randomly generate a population of variable-length binary strings (individuals). The population size and the variable-length limits are user-specified. The individual lengths are determined randomly from a lower-bound and upper-bound range of given values, e.g., [8 - 20].

### 10.2.2 Mapping

After population initialisation, the randomly generated genotypes are mapped into phenotypes. The mapping process involves the Backus-Naur Form (BNF) grammar and the rule specified by Equation 1 to produce valid phenotypes. The mapping is deterministic, meaning if the grammar remains the same, a particular genotype will always be mapped to the same phenotype.

### 10.2.3 Selection

The commonly used selection methods in GE are tournament selection and fitness-proportionate selection.

### 10.2.4 Crossover

Single-point crossover is the most widely used crossover operator in GE. This is applied similarly to genetic algorithms, except that in GE, it is used on variable-length genomes, resulting in variable-length offspring. Two parents are selected using a chosen selection method, and a crossover probability rate is used to determine if the crossover operator should be applied. If crossover is to be applied, a random crossover point r in the range [1, l - 1], where l is the maximum length of the shortest parent, is generated and used as the crossover point where the tails of each parent are exchanged.

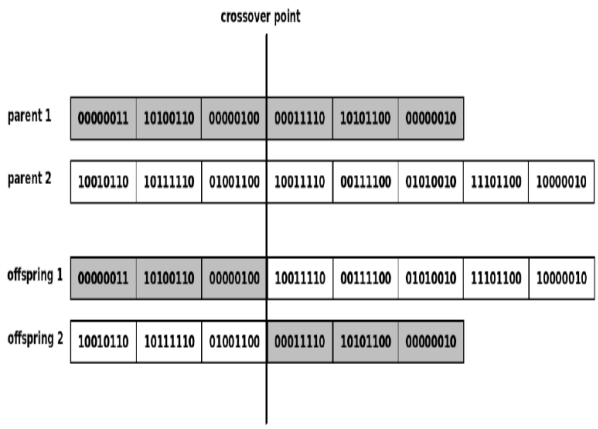


Figure 49: Single Point Crossover

### 10.2.5 Mutation

Mutation is applied in the same manner as in the genetic algorithm, e.g bit mutation.

### 10.2.6 Population Replacement

Grammatical evolution follows a similar approach to the other EAs when it comes to population replacement. Population replacement can either be generational or steady-state replacement. Elitism may also be applied in GE.

### 10.2.7 Termination

Like the other EAs, termination can occur either after a certain number of generations or if a problem-specific solution is obtained.

# 11 Swarm Intelligence

## 11.1 Introduction

We have explored the domain of evolutionary computation, specifically focusing on evolutionary algorithms. Swarm Intelligence (SI) represents another significant area that warrants our attention. SI is a field dedicated to examining the collective behaviour of social animals and insects to develop sophisticated algorithms to address complex problems. The term ”swarm” denotes a group of agents that interact with one another and their environment, adhering to simple rules to achieve intricate goals.

The concept of swarm intelligence is derived from the behaviours exhibited by social animals such as ants, bees, and termites. These animals collaborate to attain common objectives, such as locating food, constructing nests, or safeguarding their colonies. They accomplish these tasks by following elementary behavioural rules, including pheromone trails, communication, and cooperation.

Swarm intelligence algorithms are meticulously designed to emulate the behaviour of these social animals, utilising simple rules to direct the actions of a collective group of agents. These algorithms are frequently employed to resolve optimisation problems, such as determining the shortest path between two points or optimising a complex function.

Several types of swarm intelligence algorithms exist, including Ant Colony Optimisation (ACO), Particle Swarm Optimisation (PSO), and Artificial Bee Colony Optimisation (ABC). These algorithms have been successfully applied across a diverse range of applications, encompassing data clustering, image processing, and network routing.

## 11.2 Ant Colony Optimisation

Ant Colony Optimisation (ACO) is a swarm intelligence algorithm. Similar to evolutionary algorithms, ACO is a metaheuristic method that mimics the behaviour of ants to solve optimisation problems. Like evolutionary algorithms, ACO employs a population-based approach, making it a multipoint search technique. In this context, the population consists of a colony of artificial ants. In nature, ants find their way to a food source from the nest, collect the food, and return to the nest. During this process, the ants aim to minimise the distance travelled to obtain the food and return. As they move to and from the food source, ants deposit pheromones on the ground. The purpose of these pheromones is to create trails that other ants can follow to the food source and back. The strength of the pheromone trail determines the likelihood of other ants following it, with stronger trails attracting more ants.

Over time, the pheromone signal on a trail weakens unless reinforced by more ants following the trail, eventually leading to the evaporation of the pheromone. ACO algorithms mimic this behaviour by guiding a group of artificial ants to find optimal solutions to complex problems through the use of pheromone trails.

## 11.3 Artificial Ants

Ant Colony Optimisation (ACO) simulates the process by which ants collect food to solve optimisation problems. In nature, ants at each decision point choose the path that leads to food. Similarly, in ACO, artificial ants decide on which solution component to select during the process of determining a solution to the problem. The set $C$ represents the collection of solution components, and each $p_{ij}$ denotes an individual solution component. Each trail leading to a solution is associated with a pheromone parameter $T_{ij}$, and the value of this pheromone parameter is denoted by $\tau_{ij}$.

ACO identifies a solution by having ants traverse a construction graph, where the vertices or edges of the graph represent solution components. Each ant begins at a different node in the graph and incrementally constructs a partial solution by moving from one vertex to the next. Upon completing the solution construction, the pheromone is updated at each vertex or edge. Each edge in the graph has both a pheromone value and a heuristic value associated with it.

The ACO algorithm from [7] is depicted in Algorithm 11.3. The algorithm commences by setting initial parameters and pheromone values. These parameters are crucial for guiding the behaviour

**Algorithm 10** Ant Colony Optimization (ACO)[7]

```
Initialize pheromone values τ_ij and parameters
while termination condition not met do
  Generate solutions for all ants
  for each ant do
    Choose the next component C_ij based on pheromone and heuristic information
    Move to the next node in the construction graph
  end for
  Evaluate the solutions
  Update the pheromone values τ_ij based on the quality of the solutions
end while=0
```

of the artificial ants throughout the optimisation process. Parameters are associated with probabilistic rules for selecting solution components in creating solutions and rules for updating pheromone. These differ depending on the ACO algorithm. The number of ants is also a parameter. The ACO process is an iterative process which terminates when the termination criterion is met. Two commonly used termination criteria are the number of iterations and CPU time. The process begins with each ant constructing a solution. Each ant begins at a randomly selected vertex. Each partial solution is initially empty, and as the ants select components, these are added to the solutions. The ants continue selecting solution components until the solution is complete. The solution component is selected using a probability rule. The probability rule used is dependent on the ACO algorithm. An example of a probability rule is depicted below in Equation 6

The probability $p_{ij}(t)$ of an ant at node $i$ choosing node $j$ as the next node to move to is given by:

$$p_{ij}(t) = \frac{[\tau_{ij}(t)]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{k \in N_i}[\tau_{ik}(t)]^{\alpha}[\eta_{ik}]^{\beta}} \tag{6}$$

where:

- $\tau_{ij}(t)$ is the pheromone value on edge $(i, j)$ at time $t$.
- $\eta_{ij}$ is the heuristic value (often the inverse of the distance) for edge $(i, j)$.
- $\alpha$ is a parameter that controls the influence of the pheromone trail.
- $\beta$ is a parameter that controls the influence of the heuristic value.
- $N_i$ is the set of feasible nodes that can be visited from node $i$.

In Ant Colony Optimization (ACO), $p_{ij}$ represents the solution component, $\tau_{ij}$ denotes the pheromone value associated with the solution component, and $\eta_{ij}$ signifies the heuristic value. The set $N(i)$ encompasses the feasible solution component Neighbours, while $sp$ refers to the partial solution. The parameters $\alpha$ and $\beta$ are algorithm-specific, and their optimal values depend on the problem being addressed. An optional step involves applying a local search, also known as daemon actions. These actions are general procedures applied to the complete solutions and are not specific to the actions of individual ants. The final step in the ACO algorithm involves updating the pheromone values. The goal is to increase pheromone levels on trails where the solution quality is high and decrease them where the quality is low. This process begins with removing pheromones from all trails according to a pheromone evaporation rate that mirrors the natural pheromone evaporation observed in real ant trails. The specific rule used to update pheromone values depends on the particular ACO algorithm being employed. The pheromone update rule in ACO aims to increase pheromone levels on trails with high-quality solutions and decrease them on trails with poor-quality solutions. The pheromone value $\tau_{ij}(t)$ is updated as follows in Equation 7

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij} \tag{7}$$

where:

- $\tau_{ij}(t)$ is the pheromone value on edge $(i, j)$ at time $t$.
- $\rho$ is the pheromone evaporation rate ($0 < \rho < 1$).
- $\Delta\tau_{ij}$ is the amount of pheromone deposited on edge $(i, j)$ by ants.

The amount of pheromone deposited $\Delta\tau_{ij}$ is computed as:

$$\Delta\tau_{ij} = \sum_{k=1}^{m} \Delta\tau_{ij}^{k}$$

where:

- $m$ is the number of ants.
- $\Delta\tau_{ij}^{k}$ is the pheromone deposited on edge $(i, j)$ by ant $k$, which can be defined as:

$$\Delta\tau_{ij}^{k} = \begin{cases} Q/L_k & \text{if ant } k \text{ uses edge } (i,j) \text{ in its solution} \\ 0 & \text{otherwise} \end{cases}$$

- $Q$ is a constant.
- $L_k$ is the length of the solution constructed by ant $k$.

## 11.4 ACO Parameters

The algorithm involves constructing a graph representation of the problem and simulating the behaviour of ants on this graph to find the optimal solution. Here are some of the key parameters associated with ACO:

1. **Number of Ants**: The number of ants determines the ant colony size used in the algorithm. A larger number of ants can lead to a more diverse search of the solution space but may also increase the computational cost of the algorithm.
2. **Pheromone Evaporation Rate**: The pheromone evaporation rate controls the rate at which the pheromone trails on the graph evaporate over time. A higher evaporation rate means that the pheromone trails dissipate more quickly, while a lower rate allows the pheromone to accumulate over time.
3. **Pheromone Intensity**: The pheromone intensity controls the strength of the pheromone trails left by the ants on the graph. A higher intensity means that the pheromone trails have a stronger effect on the decision-making process of the ants.
4. **Heuristic Information**: The ants use Heuristic information to guide their search towards promising solutions. This information can be based on domain knowledge, such as the distance between nodes in a graph, or previous experience with the algorithm.
5. **Ant Decision Rule**: The ant decision rule determines how the ants choose their next move on the graph. This rule can be based on the pheromone trails, the heuristic information, or a combination of both.
6. **Local Search Strategy**: ACO can be combined with local search strategies to improve the quality of the solutions. Local search strategies involve making small changes to the current solution to explore nearby regions of the solution space.
7. **Termination Criteria**: Termination criteria determine when the algorithm should stop searching for a better solution. This can be based on a maximum number of iterations, a time limit, or a minimum improvement in the best solution found.

These parameters can be adjusted to optimise the performance of the ACO algorithm for a particular problem.

## 11.5 Variations of ACO

ACO algorithms generally differ in the probability rules and pheromone update rules used. ACO has several variations, each of which modifies the basic ACO algorithm in different ways to improve its performance or adapt to specific problem domains.

[label=0.]**Max-Min Ant System (MMAS)**: MMAS is a variation of ACO that uses a dynamic update rule for pheromone trail intensities. MMAS limits the amount of pheromone that can be deposited on each trail, which prevents premature convergence and ensures exploration of the search space. **Ant System (AS)**: AS is the original version of ACO, which uses a simple pheromone update rule and a static probability function for ant movement. AS is suitable for problems with a small search space and a small number of parameters. **Rank-Based Ant System (RAS)**: RAS is a variation of ACO that uses a rank-based selection strategy for ant movement. RAS ranks the potential solutions based on their fitness and selects the best ones for the next iteration, which ensures convergence to a high-quality solution. **Ant Colony System (ACS)**: ACS is a variation of ACO that uses multiple ant colonies to search for solutions. ACS introduces a global pheromone update rule that combines the pheromone levels of all ants in the system, which facilitates information sharing and improves the convergence rate. **Elitist Ant System (EAS)**: EAS is a variation of ACO that uses a selective pheromone update rule that favours the best ants in each iteration. EAS gives more importance to the best solutions found so far and prevents the loss of good solutions due to pheromone evaporation.

Overall, the variations of ACO are designed to enhance the performance of the algorithm in different problem domains and to adapt to different search space characteristics. The selection of the appropriate variation depends on the nature of the problem and the available resources.

## 11.6 Applications of ACO

Ant Colony Optimisation (ACO) has been successfully applied to a wide range of optimisation problems in various application domains. Some of the popular application domains of ACO are:

[label=0.]**Routing and Transportation**: ACO has been used to optimise routing and transportation problems, such as vehicle routing, supply chain management, and logistics planning. ACO algorithms can find the shortest path between multiple points and allocate resources efficiently. **Telecommunications**: ACO has been applied to the optimisation of telecommunication networks, such as routing in wireless sensor networks, resource allocation in mobile networks, and scheduling in satellite communication. ACO can help to reduce network congestion, improve signal quality, and minimise the cost of communication. **Manufacturing and Production**: ACO has been used to optimise production and manufacturing processes, such as scheduling, layout design, and quality control. ACO can help to minimise production time, reduce waste, and improve product quality. **Bioinformatics**: ACO has been applied to bioinformatics problems, such as protein folding, sequence alignment, and gene expression analysis. ACO algorithms can help to predict protein structure and function, identify genetic mutations, and classify biological data. **Financial Planning**: ACO has been used to optimise financial planning problems, such as portfolio optimisation, asset allocation, and risk management. ACO algorithms can help to minimise risk, maximise returns, and balance the portfolio.

Overall, ACO has proven to be a versatile optimisation algorithm that can be applied to a wide range of problems in various domains. Its ability to find near-optimal solutions and adapt to changing environments makes it a popular choice for many real-world optimisation problems.

## 11.7 Particle Swarm Optimisation

In continuous optimisation, the goal is to identify the best solution within a continuous range of possible values, enabling fine-tuning of parameters crucial in areas like engineering, economics, and

machine learning. Unlike discrete optimisation, which deals with distinct values, continuous optimisation utilises calculus-based methods, gradient descent, and numerical approaches to efficiently explore the solution space and find optimal variable values.

Particle Swarm Optimisation (PSO) is a population-based optimisation technique inspired by the social behaviour of birds flocking or fish schooling. In continuous optimisation, the goal is to find the optimal values of decision variables within a continuous range. PSO achieves this by having a group of particles (potential solutions) move through the solution space, adjusting their positions based on their own experience and the experience of Neighbouring particles. It is a metaheuristic that iteratively improves a population of candidate solutions. The candidate solutions are particles. Each particle has a location and velocity. The particles move through the search space in the direction of the optimal particles at that particular point in the search. The movement is determined by a function of their position and velocity. The movement is based on both local and global best positions at the particular point in the search. Mathematical formulae are used to move the particles. A fitness value is also associated with each particle. A problem-dependent fitness function is used to calculate the fitness value.

**Algorithm 11** Particle Swarm Optimisation (PSO) [8]

```
Initialise the swarm with random particles (positions and velocities)
Define the objective function to evaluate particles
while termination condition not met do
  for each particle do
    Evaluate the fitness of the particle
    if the fitness is better than the personal best fitness then
      Update the personal best position and fitness
    end if
    if the fitness is better than the global best fitness then
      Update the global best position and fitness
    end if
  end for
  for each particle do
    Update the velocity using:
```

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (p_i - x_i(t)) + c_2 \cdot r_2 \cdot (g - x_i(t))$$

Update the position using:

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

```
  end for
end while
Return the global best position and fitness =0
```

There are two key processes in PSO: velocity update and position update.

### 11.7.1 Velocity Update

The velocity update equation in Particle Swarm Optimisation (PSO) is a fundamental part of the algorithm, guiding the movement of particles through the solution space. This equation determines the new velocity of a particle based on its current velocity, its personal best position, and the global best position found by the swarm. Here's the equation again for reference:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (p_i - x_i(t)) + c_2 \cdot r_2 \cdot (g - x_i(t))$$

where:

- $v_i(t)$ is the current velocity of particle $i$ at time $t$.

- $v_i(t+1)$ is the updated velocity of particle $i$ for the next iteration.
- $w$ is the inertia weight, controlling the influence of the previous velocity on the new velocity. It helps balance exploration and exploitation.
- $c_1$ is the cognitive acceleration coefficient, which controls the influence of the particle's personal best position on the new velocity.
- $c_2$ is the social acceleration coefficient, which controls the influence of the swarm's global best position on the new velocity.
- $r_1$ **and** $r_2$ are random numbers between 0 and 1, providing stochasticity to the search process.
- $p_i$ is the personal best position of particle $i$.
- $x_i(t)$ is the current position of particle $i$.
- $g$ is the global best position found by the entire swarm.

The velocity update equation consists of three main components:

1. **Inertia Component** ($w \cdot v_i(t)$): This term helps the particle maintain its current direction and momentum, providing a balance between global exploration and local exploitation
2. **Cognitive Component** ($c_1 \cdot r_1 \cdot (p_i - x_i(t))$): This term encourages the particle to return to its personal best position, guiding it towards previously found high-quality solutions.
3. **Social Component** ($c_2 \cdot r_2 \cdot (g - x_i(t))$): This term attracts the particle towards the global best position discovered by the swarm, promoting collaboration and sharing of information among particles.

By updating the velocity based on these components, PSO ensures that particles explore the solution space efficiently while converging towards optimal solutions.

### 11.7.2 Position Update

The position update equation in Particle Swarm Optimisation (PSO) is a crucial component that dictates how particles move through the solution space. This equation updates the position of each particle based on its current position and its updated velocity. The equation is given by:

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

where:

- $x_i(t)$ is the current position of particle $i$ at time $t$.
- $x_i(t+1)$ is the updated position of particle $i$ for the next iteration.
- $v_i(t+1)$ is the updated velocity of particle $i$.

The position update equation essentially states that the new position of a particle is the sum of its current position and its updated velocity. This updated velocity is influenced by the particle's previous velocity, its personal best position, and the global best position found by the swarm.

By iteratively updating the positions of the particles, PSO enables the particles to explore the solution space and converge towards optimal solutions. The balance between exploration (searching new areas of the solution space) and exploitation (refining known good solutions) is achieved through the interplay of the inertia weight, cognitive component, and social component in the velocity update equation.

### 11.7.3 PSO Parameters

As in the case of genetic algorithms, various parameter values need to be set for the PSO algorithm. The best values for these parameters are problem-dependent. Parameter tuning has to be performed to determine the best values to use. Parameters for the PSO algorithm include:

[label=0.]**Swarm Size**: The swarm size is the number of particles used in the algorithm. A larger swarm size can lead to a more diverse search of the solution space, but may also increase the computational cost of the algorithm. **Maximum Velocity**: The maximum velocity is a parameter that limits the maximum change in velocity of each particle. This helps to prevent the particles from overshooting the global best position and increases the convergence rate of the algorithm. **Inertia Weight**: The inertia weight is a parameter that balances the exploration and exploitation of the search space. This weight factor controls the balance between the current velocity and the difference between the current position and the global best position. **Acceleration Coefficients**: The acceleration coefficients determine the influence of the personal best and global best positions on the movement of the particles. These coefficients control the rate at which the particles move towards the best solutions found by themselves and by the swarm as a whole. **Neighbourhood Topology**: The Neighbourhood topology determines how the particles communicate and share information with each other. The topology can be defined as a global, local, or hybrid network. **Local Search Strategy**: PSO can be combined with local search strategies to improve the quality of the solutions. Local search strategies involve making small changes to the current solution to explore nearby regions of the solution space. **Termination Criteria**: Termination criteria determine when the algorithm should stop searching for a better solution. This can be based on a maximum number of iterations, a time limit, or a minimum improvement in the best solution found.

These parameters can be adjusted to optimise the performance of the PSO algorithm for a particular problem. However, finding the optimal values for these parameters can be challenging and may require a trial-and-error approach or the use of optimisation techniques.

### 11.7.4 Variations of PSO

Variations of the standard PSO algorithm exist. These variants differ in terms of:

- Initial values of positions and velocities
- Update rules, e.g., updating the local and global position only after the swarm is updated
- Velocity update equation

[label=0.]**Constriction Factor PSO**: This variation introduces a constriction factor that limits the maximum change in velocity of each particle. This helps to prevent the particles from overshooting the global best position and increases the convergence rate of the algorithm. **Inertia Weight PSO**: Inertia weight PSO modifies the velocity update equation by adding an inertia weight term that balances the exploration and exploitation of the search space. This weight factor controls the balance between the current velocity and the difference between the current position and the global best position. **Cooperative PSO**: Cooperative PSO introduces multiple sub-swarms that work independently and exchange information periodically. The sub-swarms explore different areas of the search space, and the information exchange helps to improve the overall performance of the algorithm. **Hybrid PSO**: Hybrid PSO combines the PSO algorithm with other optimisation algorithms, such as genetic algorithms or simulated annealing. This approach helps to improve the global search capabilities of the PSO algorithm and overcome its limitations. **Multi-objective PSO**: Multi-objective PSO extends the PSO algorithm to solve optimisation problems with multiple objectives. This variation uses Pareto optimisation to find the set of non-dominated solutions that provide a trade-off between different objectives.

# 12 Adaptation and Hybrid Methods

## 12.1 Adaptation

An adaptive evolutionary algorithm is a type of evolutionary algorithm that incorporates adaptation mechanisms to dynamically adjust its parameters during the optimisation process. Unlike traditional evolutionary algorithms, which use fixed parameters, adaptive evolutionary algorithms can adjust their parameters in response to changes in the problem landscape or population behaviour, allowing them to explore and exploit the search space more effectively. Adaptive evolutionary algorithms typically include one or more adaptation mechanisms, which can be categorised into two types: self-adaptation and co-evolution.

**Self-adaptation mechanisms** adjust the algorithm's parameters based on the performance of the algorithm itself, while co-evolution mechanisms adapt the algorithm's parameters in response to the behaviour of the population being evolved. Examples of self-adaptation mechanisms include mutation step size adaptation, crossover probability adaptation, and population size adaptation. These mechanisms adjust the algorithm's parameters based on the current state of the population, allowing the algorithm to explore the search space. **Co-evolution mechanisms**, as stated, adapt the algorithm's parameters based on interactions between the individuals in the population. Examples of co-evolution mechanisms include fitness sharing, where individuals are penalised for being too similar to others in the population, and niching, where individuals are encouraged to explore different regions of the search space to avoid crowding. Overall, adaptive evolutionary algorithms are powerful optimisation tools that can adapt to changing problem landscapes and population behaviour, allowing them to more effectively explore and exploit the search space

## 12.2 MultiObjective Optimisation

When faced with optimisation challenges involving multiple, often competing goals, Multiobjective Evolutionary Algorithms (MOEAs) offer a powerful approach. Unlike traditional optimisation, which focuses on a single objective, MOEAs aim to find a set of optimal solutions that represent the balance between these conflicting objectives. These solutions, forming the 'Pareto front,' are those where improving one objective means sacrificing another. By utilising evolutionary principles, MOEAs efficiently explore the vast solution space and provide decision-makers with a range of viable options, particularly useful in fields like engineering, finance, and environmental management. An example of a multi-objective (MO) problem is the design of a bridge. We want to (1) Minimize Cost, (2) Maximize Strength, (3) Maximize Durability, and (4) Minimize Environmental impact. These are conflicting desires which require balancing.

## 12.3 Hybrid Algorithms

We hybridise algorithms to combine the strengths of different algorithms and overcome their weaknesses. No single algorithm is universally effective for all optimisation problems. Therefore, by combining multiple algorithms, we can create a more powerful and robust optimisation method capable of finding high-quality solutions to a wider range of optimisation problems. Here are some benefits of algorithm hybridisation:

- Improved solution quality: Hybridisation can lead to improved solution quality by combining the strengths of different algorithms. For example, a genetic algorithm might excel at global exploration while a local search algorithm might excel at exploiting promising regions of the search space. Combining these two algorithms can result in a more effective optimisation method that achieves better solution quality than either algorithm alone.

- Faster convergence: Hybridisation can speed up the optimisation process by combining different algorithms that converge at different rates. For example, a genetic algorithm might have slow convergence but provide good diversity, while a local search algorithm might converge

faster but get stuck in local optima. Combining these two algorithms can lead to faster convergence and better exploration of the search space.

- Increased robustness: Hybridisation can increase the robustness of an optimisation method by making it less sensitive to changes in the optimisation problem or the algorithm parameters. By combining different algorithms, we can reduce the risk of being stuck in local optima or trapped in a suboptimal region of the search space.
- Better scalability: Hybridisation can improve the scalability of an optimisation method by enabling it to handle larger and more complex optimisation problems. By combining different algorithms, we can leverage their complementary strengths to handle different aspects of the problem, resulting in a more efficient and effective optimisation method.

Overall, hybridisation is a powerful tool for creating more effective optimisation methods that can tackle a wide range of optimisation problems.

### 12.4 Hybridisation of Evolutionary Algorithms

Algorithm hybridisation in evolutionary algorithms involves combining two or more different algorithms to create a more effective optimisation method. The goal of hybridisation is to leverage the strengths of different algorithms and overcome their weaknesses, resulting in a more powerful and efficient optimisation method. There are several ways to hybridise evolutionary algorithms, including:

1. Combining crossover and mutation operators from different evolutionary algorithms: For example, a genetic algorithm might use a crossover operator from one algorithm and a mutation operator from another algorithm.
2. Combining different selection strategies: Different selection strategies, such as tournament selection and roulette wheel selection, can be combined to improve the diversity of the population and prevent premature convergence.
3. Using local search methods: Local search methods, such as hill climbing or simulated annealing, can be used in conjunction with evolutionary algorithms to refine solutions and improve convergence.
4. Incorporating machine learning techniques: Machine learning techniques, such as neural networks or decision trees, can be used to guide the search process and improve the quality of the solutions.

Hybridisation is particularly useful for tackling complex optimisation problems that may require a combination of different techniques to solve. By combining the strengths of different algorithms, hybridised evolutionary algorithms can provide a more powerful and efficient optimisation method, resulting in faster convergence and better quality solutions. Examples of hybridised evolutionary algorithms include memetic algorithms, which combine genetic algorithms with local search methods, and particle swarm optimisation with local search (PSOLS), which combines particle swarm optimisation with local search methods.

## A References


1. Aggarwal, C.C. and Aggarwal, C.C., 2021. An introduction to artificial intelligence. Artificial Intelligence: A Textbook, pp.1-34.
2. Russell, S.J. and Norvig, P., 2016. Artificial intelligence: a modern approach. pearson.
3. Lourenço, Helena R., Olivier C. Martin and Thomas Stützle. ”Iterated local search.” Handbook of metaheuristics. Boston, MA: Springer US, 2003. 320-353.

4. Knapsack Problems: Algorithms and Computer Implementations (Wiley Series in Discrete Mathematics and Optimisation)

5. Holland, J. H. (1992). Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. MIT Press.

6. Koza, J. R. Genetic programming as a means for programming computers by natural selection. Statistics and computing 4, 2 (1994), 87–112.

7. Langdon, W. B., and Poli, R. (2013). Foundations of genetic programming. Springer Science and Business Media.

8. O'Neill, M., and Ryan, C. (2001). Grammatical evolution. IEEE Transactions on Evolutionary Computation, 5(4), 349-358.

9. M. Dorigo and K. Socha, "An introduction to ant colony optimization," IRIDIA, Tech. Rep., 2007.

10. Kennedy, J. and Eberhart, R., 1995, November. Particle swarm optimization. In Proceedings of ICNN'95-international conference on Neural Networks (Vol. 4, pp. 1942-1948). ieee.